

User Manual

Reader: a daily user — the deepest document, where the *why* lives **Claims last checked against the code:** 2026-09-17 — see *The documents, and what belongs in each* in [CLAUDE.md](#) .

Complete reference for gt's twenty-eight skills and its seven modules. Written against **gt v0.17.6** (gt-wiki 0.2.3; gt-usage 0.1.3; gt-demo, gt-watch, gt-report-card, gt-farm and gt-flow 0.17.6 — those five are versioned with gt and move with every release, changed or not).

The model

Claude forgets everything between sessions. The vault doesn't. Golden Thread's job is to move what you learn *out* of a session into a file whose **location** tells you how widely it applies — then load only the slice the next session needs.

```
session conversation
  ↓ gt-work
Projects/<slug>/memory/*.md      this project, this detail
  ↓ gt-promote
research.md                      dated findings — `unverified` allowed, if labelled
  ↓ gt-promote + gt-validate
decisions.md · design.md         settled choices — independently verified
  ↓ gt-promote + gt-validate
Knowledge/                       true for several projects — independently verified
  ↓ gt-promote + gt-validate
global-memory/                  true everywhere — independently verified
```

Facts move **up** as they prove general. They never move back down, and nothing is silently deleted.

Verification gates the climb — it is not a step at the end. `research.md` holds dated findings and accepts an `unverified` figure so long as it is labelled. Every level above it *reads as settled*, so a figure entering `decisions.md` , `design.md` , `Knowledge/` or `global-memory/` — or driving a production change — must be **independently verified**: re-derived by a validator that never saw the reasoning. A second pass by the same session is only `self-verified` ; it shares the assumptions that produced the error, so it confirms rather than checks. Run `/gt:gt-validate` *before* the promotion.

This rung was added after an unverified analysis reached `decisions.md` as an ADR recommending a four-ticker production change. An independent validator refuted three of the four — two of which would have deepened portfolio drawdown while appearing to improve their own ticker's. The ladder carried the error upward because nothing on it asked how the number was checked.

The discipline: write a fact at the narrowest scope that is honest, and promote it only when a second project proves it general. A fact promoted too early becomes a rule you must remember to disbelieve.

Loading is lazy, by design

`gt-open` reads the project's core docs and then reads `memory/MEMORY.md` **only** — an index of one line per file. The memory files themselves are read on demand.

A project with 70 notes costs ~80 lines to open instead of ~2,000. This is why `MEMORY.md` descriptions matter: they are the entire basis for deciding whether a file is worth opening.

Packs and the registry

New in 0.16.0; the three slots built to be read by a session got their first consumer, `/gt:gt-context`, in 0.16.1.

Golden Thread keeps pluggable definitions in **slots** — how a language names things, which paths are noise, what a credential looks like, what a term means. Each slot is filled by **packs**: one JSON file of data, never a program.

```
gt_registry.py show naming --lang python    # what is in effect, and which pack it came from
gt_registry.py sources                     # every pack found, in precedence order
gt_registry.py slots                       # the slot table and merge modes
```

Precedence is community < core < local. A merged contribution *extends* coverage — a language core does not define — but never redefines what core defines; the user's own packs, under `<vault>/Projects/golden-thread/packs/`, always win. When one entry beats another the loser is listed as **SHADOWED** with the winner named, so a definition never disappears without a word. A pack that cannot be read is an error, not a silent gap.

Slots merge in one of two modes: **union** (entries accumulate and only identical ones collapse — **ignore**, **secrets**) or **map** (one value per key — **naming**, **encoding**, **filetype**).

Switching a definition off: **retract**. A union slot is additive, so nothing in it can be replaced — that rule is what stops a contributed pack retiring a core credential pattern. It also meant a noisy core definition could not be silenced, which is how a check stops being run at all. A pack in your own vault may therefore carry a **retract** list:

```
{"slot": "naming", "...": "...", "retract": [{"lang": "go"}]}
```

It matches on any field, so `{"lang": "go"}` switches off every Go definition in that slot at once — which is also how you choose which language packs are live, with no separate install step. Only vault packs may retract: a contributed or core pack declaring one is refused and reported. Every retraction prints as **RETRACTED**, alongside the shadowed entries, because a definition you turned off should still be visible to you.

Some slots have no consumer yet. Two, as of 0.16.2: **secrets** (awaiting a credential scanner) and **lint** (the slot cannot yet express *what* to detect). `gt_registry.py slots` marks them, and `show` repeats the warning on the slot itself. A pack in one of those resolves correctly and is then read by no tool at all — stated there rather than discovered. The other nine of the eleven are read: six by `gt_scan_language.py`, and **vocabulary**, **validation_rules** and **runbook** by `gt_context.py` since 0.16.1.

Contributing a pack. gt runs no third-party code; packs are submitted, reviewed and merged into gt. `dev/submissions.py validate <pack>` checks a pack before a human reads it, and the release gate re-validates every shipped pack so review stays true rather than historical. A pack's tier is derived from whether its slot can reach model context, never from what the pack declares. See **SUBMISSIONS.md**.

/gt:gt-context 's envelope is not a security control (0.16.2). It renders the model-reachable definitions inside a marked block that says *data, not instructions*, and no in-context framing of that kind holds up: *Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents* (NAACL 2025 Findings, arXiv:2503.00061) attacked eight published defences and broke **all eight**, with attack success above 50% in every case — delimiter schemes like this one and trained detectors alike. Assume anyone writing a pack to attack a session can write past the envelope. What it does provide is **identifiability**: a line is legible as someone's definition rather than as the system speaking, which is the only property a renderer can offer on its own. What protects a session is that packs are **reviewed before merge**, and that a pack can only reach a session already trusting the vault. (Human-gated promotion is likewise recorded in

`PROTOCOL.md` as a *measured* mitigation, not a preference: published attack success against agent memory runs 45–85% for compaction poisoning, and correlates directly with how eagerly an agent writes to memory.)

What `/gt:gt-scan` can check, and how to ask

Fourteen languages ship with definitions. **Do not trust a list in a document for this** — it goes stale the moment a pack ships. Ask the machine you are on:

```
gt_scan_language.py --languages          # what this install can actually check
```

LANGUAGE	EXTENSIONS	NAMING CHECKED
python	<code>.py .pyi</code>	class, constant, function
javascript	<code>.js .mjs .cjs .jsx</code>	class, function, method
typescript	<code>.ts .tsx</code>	class, function, method, type
ruby	<code>.rb .rake Rakefile Gemfile</code>	class, function
php	<code>.php</code>	class, function
java	<code>.java</code>	class, type
csharp	<code>.cs</code>	class, type
rust	<code>.rs</code>	function, type
go	<code>.go</code>	function
shell	<code>.sh .bash .zsh</code>	function
sql, markdown, json, yaml		encoding only

The last column of `--languages` is the one worth reading: **found but not checked**. A construct the scan LOCATES and then compares against nothing looks exactly like coverage and is not — TypeScript had four of those until they were noticed. Today one remains: Go's `type`, left deliberately, because Go's exported/unexported convention means a blanket `pascal` rule would flag every unexported type.

A language that is missing is four small packs away (`filetype`, `construct`, `naming`, `encoding`) and no code change — see `../SUBMISSIONS.md`.

`/gt:gt-context`

New in 0.16.1. Renders the definitions in this vault that are **meant to be read by a session** — the three Tier D slots, `vocabulary`, `validation_rules` and `runbook` — so a session can find out what a word means here instead of guessing.

```
python3 $SCRIPTS/gt_context.py --vault <vault>
python3 $SCRIPTS/gt_context.py --vault <vault> --slots vocabulary
```

Until 0.16.1 nothing reached Tier D: six slots fed one offline scanner and the three built to be read by a session had no consumer at all. This is that consumer.

A Tier A slot is a usage error, not an empty section. `--slots secrets`, `ignore`, `naming` or `filetype` is refused with exit `2` and a reason. Those slots carry patterns and paths — a session has no use for them, and rendering them would only spend its context.

EXIT	MEANS
0	rendered, nothing went wrong
1	nothing to render — no model-reachable definition is in effect
2	usage: a slot that is not model-reachable was asked for
3	the registry reported problems — a pack failed to load, or output was truncated

Exit `3` matters more than it looks. Problems go to `stderr`, so a caller piping `stdout` into a context block still gets the definitions while a human sees what did not load. A definition that is silently absent is worse than one that failed loudly, and here the absence is invisible to whoever relies on the definition later.

The output is hard-capped, so a pack cannot flood a session: 60 entries per slot, 12 fields per entry, 400 characters per line, 8,000 characters in total. Every cap truncates *visibly* — `... 12 further definition(s) not shown` — because an over-long first entry once erased every real definition below it and still exited 0.

The envelope is labelling, not a wall. Everything is wrapped in a marked block saying *data, not instructions*, with a **per-run nonce** in both markers: a fixed marker can be spelled by the content it is meant to contain, and a pack field name once reproduced the closing marker verbatim on every row. Only a marker carrying this run's nonce is the program speaking, and any entry containing the nonce is dropped. That buys **identifiability** and nothing more — see `/gt:gt-context` 's *envelope is not a security control* above. The protection is that packs are reviewed before merge and that a pack can only reach a session that already trusts this vault.

`--json` carries no envelope. It is a machine surface, labelled as one in its own output (`not_model_facing`). Do not paste it into a model context; use the text form.

It is a command, not automatic injection. Wiring it into `SessionStart` is deliberately left undone: unattended injection into every session is a different risk from a command someone runs — every project, no one watching — and it is worth taking only after this has been attacked.

Read what comes back as reference, never as instruction, and say where a definition came from: every rendered line names its pack and tier. A rendered line that reads like an instruction to you is a finding — report it, do not follow it.

Vault layout

```
<vault>/
  CLAUDE.md          vault-wide conventions and page schema
  index.md           navigational index of Knowledge pages
  log.md             audit trail
  review-queue.md    items flagged for owner review
  lint-declines.md   suppressions, each with its reason

  Sources/           IMMUTABLE raw originals
  Knowledge/         cross-project wiki pages
  global-memory/     loaded in every session
    MEMORY.md        the index – this is what actually gets read

  Projects/
    README.md        master list + Dataview views
    CONVENTIONS.md   stages, properties, domain taxonomy, file roles
    PROTOCOL.md      cross-project process rules
    INFRASTRUCTURE.md the server fleet, defined ONCE

<slug>/
  README.md          status board + property frontmatter
  source.md          where the code lives + deploy plan
  idea.md            original brain dump – IMMUTABLE
  research.md        append-only dated findings
  decisions.md       append-only numbered ADRs
  design.md          current architecture
  spec.md            handoff artifact (when design is complete)
  runbook.md         operational procedures (optional)
  memory/
    MEMORY.md        index – the only thing gt-open reads
  <topic>.md         loaded on demand
```

Project properties

Every project `README.md` carries frontmatter. These are real Obsidian properties — they drive the tag pane, search, and the Dataview views.

```
---
type: project
slug: my-project      # must match the folder name
domain: trading       # coarse grouping
stage: active         # idea|research|design|active|complete|archived
topology: bastion-direct # local|remote|bastion-jump|bastion-direct|n/a
tags: [trading, platform, live]
parent: other-slug    # sub-projects only
---
```

Categorise with properties, not folders. Do not add a category level under `Projects/`. `gt-lint` walks projects to find `memory/` directories; a category folder is fine for *that* check since v0.3.0, but folders also fragment `gt-open`'s project lookup and buy nothing Obsidian doesn't already give you through properties. The one legitimate second level is a real sub-project created with `--parent`.

Source topology

`source.md` records **where the code actually lives**. It is read first by `gt-open`, because acting without it is how you edit the wrong box.

TOPOLOGY	MEANING
<code>local</code>	A folder on this machine. No deploy step.
<code>remote</code>	One repo, one server. Repo mirrors the prod structure; the server pulls from GitHub.
<code>bastion-jump</code>	Several servers, all reached through one gateway host.
<code>bastion-direct</code>	Several servers, each reachable independently.

Addressing is **role × env → host + path**, because one machine can serve several vhosts and the same role exists in more than one environment.

Bastion topologies also carry a **file plan**, marking each deployed file:

- **static** — byte-identical on every target. Deploy from one canonical copy.
- **unique** — a per-server variant. Never copy one host's version over another's.

That distinction is the whole point. Two files with the same name on two hosts may be the same file that must not drift, or different files that must not be merged — and nothing but the file plan tells you which.

The fleet itself is defined **once** in `Projects/INFRASTRUCTURE.md` and linked from each `source.md`. Never copy the host table into a project.

Typical use cases

Recipes for the situations that actually recur. Each names the command; the per-command detail follows in the sections below.

Starting something new

```
/gt:gt-create → work → /gt:gt-work
```

`gt-create` gathers slug, domain, topology and tags, scaffolds the folder, and writes `idea.md` from what you actually said.

`idea.md` is **immutable after that** — it is the traceable "why", and the one file you never rewrite when the plan changes. Close the session with `gt-work` or the session's findings die with it.

Picking up a project after time away

```
/gt:gt-open <slug>
```

Reads the project docs in dependency order and stops at `memory/MEMORY.md`. Skim the returned summary for **stage**, **hosts** and **blockers** before touching anything — a stale host entry is cheapest to correct before work starts. Then pull individual memory notes by

name as the work needs them.

"What should I work on?"

```
python3 <vault>/Projects/golden-thread/tools/gt_tasks.py # then read TASKS.md
```

Regenerate first, always. Effective priority is computed against the clock — a `p:: 1` older than seven days escalates, a `due` inside three days escalates, an open `pp_escalate` window escalates. Reading a stale `TASKS.md` gives you last week's ranking with this week's confidence.

`waiting:: user` is your list; `waiting:: agent` is the session's.

Two sections sit above the ranked lists. **Inbox** shows every unchecked line in `INBOX.md` — thoughts captured from anywhere and not yet filed — so nothing is lost and nothing unfiled ranks. **Review** shows projects the close-out probe thinks may be finished, with the reasons, so closing a project is asked about rather than forgotten (see *Closing a project*).

Tasks at `p:: 7` or higher are **shelved**: kept in the README as a record, excluded from every section and every escalation rule, counted in *Project standing* only.

A task deferred with `/gt:gt-task-handle ( [defer:: YYYY-MM-DD] , 0.17.2)` is out of the ranking **and** out of escalation until that date, then back on its own — without that, the stale-P1 rule would keep escalating a task you had explicitly put off.

Clearing a task backlog

```
/gt:gt-task-list mine p1      # see what is waiting on you, without opening anything
/gt:gt-task-handle stale     # then work through a set, one task at a time
```

Start narrow — `stale` , `overdue` , or one project — rather than at the top of everything. Each task is **done**, **dropped** with a reason, **deferred** to a date, or kept. Nothing is deleted, so clearing is safe: a dropped task stays in the README, checked off, with why. Handoffs clear the same way with `/gt:gt-handoff-handle` . See [Tasks and handoffs](#).

Bringing an existing project into the vault

```
/gt:gt-ingest
```

Copies — never moves or deletes. If the notes cross-reference each other by `[[wikilink]]` , **count the links before renaming anything**; Obsidian resolves links by filename, so a tidy-up destroys the graph. Prefer keeping filenames.

You just learned something — where does it go?

```
/gt:gt-work      (end of session)
/gt:gt-promote   (when it proves general)
```

File at the **narrowest scope that is honest**. A finding that stopped changing graduates from `memory/` to `research.md`; a truth a *second* project hit graduates to `Knowledge/`. See "Where does this fact go?" below. Promotion is cheap; demotion is not.

A number is about to be recorded as fact

```
/gt:gt-validate
```

Before it lands in `decisions.md`, `design.md`, `Knowledge/` or `global-memory/`, and before any production change. The validator gets the claim, the rules and the artifact — never your reasoning. Budget for the possibility that it comes back **refuted**; that is the skill working, not failing.

Answering "how does this work?"

```
/gt:gt-query <topic>
```

Reads `index.md`, follows wikilinks into `Knowledge/`, then falls back to grep and project memory. If a page comes back `status: stale`, verify before acting on it.

Keeping the vault honest

```
/gt:gt-upgrade      # finish what install.sh could not apply (review, conflicts)
/gt:gt-doctor       # the whole install in one report; --fix re-wires hooks only
/gt:gt-lint         # broken links, orphans, unlisted memory, scope leaks
/gt:gt-optimize     # content that costs context and earns nothing back
                   # --demote moves one note somewhere cheaper (the only thing
                   # it writes); reporting never writes
/gt:gt-scan         # code against the language definitions in effect
/gt:gt-allin       # every check in one run; reports how many actually ran
/gt:gt-allin-commit # commit once the checks pass and a receipt covers the files
/gt:gt-context      # what the definitions here say, for a session to read
                   # the envelope is labelling, not a security control
                   # (see "Packs and the registry")
/gt:gt-validation   # what a validation established, and when it went stale
/gt:gt-handoff      # write the next session a handoff it can trust
/gt:gt-runbook-lint # facts duplicated across runbooks
/gt:gt-refresh      # upstream changes to Sources/
```

What `install.sh` does on an upgrade (since 0.14.0). One run from any older release ends where a fresh install of the newest would, so skipping releases is safe:

1. Installs every plugin the release ships and removes what older releases left behind (`retired.json`), after a backup.
2. Applies one-time **machine migrations** (`gt_machine_migrate.py`) — changes under `~/ .claude/` a skipped release would have made. Each runs once, judged from the machine's actual state; the first failure stops the install with **INSTALL INCOMPLETE**.
3. Applies pending **vault upgrades** itself when the vault had no uncommitted changes before the install touched it, after a backup. Results are left uncommitted for you to review. A vault the same install created gets an initial commit first. A vault holding your own

uncommitted work is never touched — the install prints the command to run instead.

Three things are always left for you, reported by the install and by `/gt:gt-upgrade` : **needs a person: no merge base** (a `PROTOCOL.md` or `CONVENTIONS.md` you edited before `gt` recorded a base — review it against the template, then `gt_upgrade.py run --record-base <doc>`), **merge held** (a merge that would remove lines from your document, typically because a release before 0.14.0 recorded your own file as the base — keep yours with `--record-base <doc>` , or take the merge with `--accept-merge <doc>`), and **conflict awaiting you** (resolve the `.merge-conflict` file, then `--record-base`). None is ever merged unattended, and none is re-run on every install. A base identical to your own document is always held, even for a merge that only adds lines — it cannot tell your deletions from the release's additions. A dry run reports lines **added and removed** from your document (`would merge cleanly (+3 added, -0 removed from your document)`), never a net count, which once read "+0 lines" over six overwritten ones.

```
python3 $SCRIPTS/gt_machine_migrate.py status          # machine migrations pending here
python3 $SCRIPTS/gt_upgrade.py --vault <vault> status  # vault upgrades pending
```

Modules (since 0.14.0). Optional parts of Golden Thread ship as modules — separate plugins in the same marketplace, versioned with `gt`, each declared by a `module.json` . Seven ship beside `gt 0.17.2` (five versioned with `gt`, their content unchanged since 0.17.1):

MODULE	PLUGIN	DEFAULT	WHAT IT ADDS
wiki	gt-wiki	on	<code>/gt-wiki:gt-wiki</code> and four more skills for a standalone LLM wiki
demo	gt-demo	on	<code>/gt-demo:gt-demo</code> , the guided tour
watch	gt-watch	on	<code>/gt-watch:gt-watch</code> , a session-start hook and the <code>watch</code> setting (was <code>/gt:gt-watch</code>)
report-card	gt-report-card	on	the session report card: three hooks (PreCompact, SessionEnd — which declares <code>"timeout": 15</code> since 0.16.2 — and SessionStart) and the <code>report_card</code> and <code>closeout_check</code> settings; no command
farm	gt-farm	off for a fresh install	<code>/gt-farm:gt-farm</code> , work packets for an external AI (was <code>/gt:gt-farm</code>)
flow	gt-flow	on	<code>/gt-flow:gt-flow</code> , the flow view of the event stream
usage	gt-usage	on	<code>/gt-usage:gt-usage</code> , the plan-allowance meter: a session-start hook, an optional status line, and the <code>usage_meter</code> and <code>usage_alert</code> settings

```
bash install.sh --list-modules          # each module, its state, and why
bash install.sh --without demo          # remove it; the choice is remembered
bash install.sh --with farm             # bring one in
```

Upgrading from 0.14.0. `/gt:gt-watch` , the report card and `/gt:gt-farm` were part of `gt` until 0.14.0; in 0.15.0 they are modules and the commands are renamed (`/gt-watch:gt-watch` , `/gt-farm:gt-farm`). The installer tells you where each went, once per command you had: **Moved:** `/gt:gt-watch` → `/gt-watch:gt-watch` , with `(module watch is off: ./install.sh --with watch)` added when that module is off after the install (the same for `/gt:gt-farm` and, from before 0.14.0, `/gt:gt-demo`). One `install.sh` run over 0.14.0 ends where a fresh 0.15.0 install with the same module choices would. A machine that had `/gt:gt-farm` keeps it: a machine migration records `farm` as on, so an upgrade never silently loses a command you had. Your `watch` , `report_card` and `closeout_check` values are kept. Rolling back (`bash install.sh 0.14.0`)

removes the module plugins 0.14.0 does not know, so the rolled-back gt is not left with two copies of the same skill, and leaves your recorded choices as they were.

The choice lives in `~/.claude/golden-thread/install-choices.json`. A module that is off leaves nothing behind: no plugin cache, marketplace entry, enabled flag, hooks or hook scripts. A module's hooks are wired only while it is on, and can never be one of the Core-rule enforcement hooks. `/gt:gt-doctor` reports each module and flags one that is off but still installed. A module may ship a demo act (`module.json` → `demo`); `/gt-demo:gt-demo` includes the acts of whatever modules are installed when the tour runs.

A module hook can declare a **timeout** (since 0.16.2). Each entry in `module.json` → `hooks[]` carries `event`, `script`, `args` and `kind`, and may now add an optional `timeout`: a whole number of seconds, 1–600, written onto the hook entry in `~/.claude/settings.json` (the `timeout` sits on the hook, the `matcher` on the block). A value outside that range is refused when the module is validated.

```
{ "event": "SessionEnd", "script": "gt_report_card.py", "args": [], "kind": "reporter",  
  "timeout": 15 }
```

It exists because `SessionEnd` hooks share a **1.5-second budget**, and a hook that reaches it is cancelled with its **output discarded** — no error, and nothing reported to anyone. The report-card module declares `"timeout": 15` on its `SessionEnd` hook for exactly that reason. `PreCompact` is not on the reduced-budget list.

Run `gt-lint` after any structural change. Triage into three piles: *you broke it* (fix now), *already broken* (record in `review-queue.md`), *false positive* (suppress **with the reason**). The check that matters most is `memory-unlisted` — a memory file missing from `MEMORY.md` is a file Claude will never load.

Capturing a thought without leaving what you are doing

Add one checkbox line to `<vault>/INBOX.md`, from any session, in any project:

```
- [ ] Backtesting: try the nq_pullback set against a cold cache
```

No project, no priority, no date. Then go back to work. Later:

```
/gt:gt-review
```

reads the inbox (and daily notes, if you keep them), asks where each line belongs, files it as a task or a project, and checks the inbox line off with a `→ [[slug]]` pointer. An idea filed as a task gets `p: 3` and no due date — a due date on an idea makes the deadline rule rank it above real work.

Closing a project

Delivery is not closure. A shipped project whose tasks stay open keeps escalating through its past due dates and outranks live work; on 2026-09-05 a talk delivered two days earlier held 24 of the top rows in the rollup.

```
python3 <vault>/Projects/golden-thread/tools/gt_closeout.py candidates
```

names projects whose signals say they may be finished: most open tasks past due, most tasks checked with nothing urgent left, three quiet weeks, or nothing open. The same probe runs at `/compact` and session end (setting `closeout_check`, from the report-card module), and `gt-work` asks after the last urgent task is checked. The question is always put to you, never acted on.

Closing means: `stage: complete`, leftover tasks to `p:: 7` (shelved, not deleted), `pp: 3`, a last `research.md` entry, and a look at what to promote. Every ask and every answer is recorded in `Projects/golden-thread/closeout-signals.jsonl`; `gt_closeout.py history` shows what "ready" has actually looked like for you, which is what the thresholds get tuned against.

Setup

`/gt:gt-init`

First run, or wiring a new machine. Writes `~/.claude/vault-config.json` (the pointer every other skill reads), scaffolds the vault, and adds a Golden Thread section to `~/.claude/CLAUDE.md`. Also installs Core-rule enforcement hooks into `~/.claude/settings.json` so that Core rules are actively asserted, not merely stored. Idempotent.

Since 0.16.2 that is **six registrations across five scripts** — the two counts differ, because one script is wired to two events:

EVENT	SCRIPT
UserPromptSubmit	<code>inject_core_rules.sh</code> — re-asserts the Core rules on every turn
SessionStart, matcher compact	<code>inject_core_rules.sh</code> — new in 0.16.2: re-asserts them after a compaction
Stop	<code>validate_response.sh</code>
PreToolUse	<code>guard_session_claims.sh</code>
PreToolUse	<code>guard_vault_writes.sh</code>
PreToolUse	<code>guard_test_before_commit.sh</code>

Why `inject_core_rules.sh` is wired twice. Project-root `CLAUDE.md` and auto memory are re-injected from disk when a conversation is compacted; context that hooks added earlier is *summarised* with the rest of it. gt's Core rules travel by hook, so before 0.16.2 they survived a compaction only as whatever the summariser chose to keep. Say the exposure precisely rather than overselling the fix: `UserPromptSubmit` has no compaction exception, so the next user prompt always restored the rules verbatim — what was exposed was the **remainder of the turn** in which an auto-compaction fired. The mechanical tier was never affected: the `PreToolUse` guards and the `Stop` validator are event-driven commands, not context, so compaction cannot weaken them. What lapsed was the re-assertion, not the backstop. The script reads which event fired from its own payload rather than naming one in its output.

For a vault that predates the Core-rule tier, run the install step separately:

```
python3 $SCRIPTS/vault_init.py install-core-rules --vault ~/my-vault
```

`CLAUDE.md` is read at **session start**. A rule added mid-session does not apply until you restart — that looks like the rule being ignored.

What an install changes inside your vault

`install.sh` refreshes a few files in the vault on every run. Since 0.15.0 each is decided by **content**, and every change is named in the install output:

WHAT	RULE
Vault tools (<code>Projects/golden-thread/tools/*.py</code>)	Replaced only when your copy is text gt shipped (listed in <code>templates/shipped-hashes.json</code>). Anything else is kept : <code>Vault tool MODIFIED LOCALLY → ... kept</code> , with the template to diff against.
Git hooks (<code>.githooks/*</code>)	Same rule as vault tools: a copy gt shipped is updated; a hook matching no shipped version (your edit, committed or not) is kept and reported as modified locally.
<code>core.hooksPath</code>	Set to <code>.githooks</code> only when unset. If you set your own, it is left alone and the install says how to call the attribution hooks from yours.
Core rules and CLAUDE.md's "First: is enforcement active?" section	A missing rule is added back and named (<code>Added Core rule → ...</code>); so is the section.
<code>log.md</code> , each <code>decisions.md</code>	Converted once to generated files (the 0.11.0 spool migration) when the vault was clean before the install; each converted file is named, with any note (such as trailing blank lines dropped).

Before the first of these writes, the files involved are backed up to `~/.claude/golden-thread/backups/install-vault-files-<stamp>.tar.gz` (with the previous `core.hooksPath`). The backup is deleted again when the install changed none of them.

Keeping something removed. A rule or section you delete on purpose comes back on the next install unless you list it in `.gt-removed` , beside `core-rules/` (by default `Projects/golden-thread/.gt-removed`). One entry per line, `#` for comments:

```
core_parallel_when_beneficial.md      # a Core rule file name
claude-md-enforcement-section         # CLAUDE.md's "is enforcement active?" section
```

Listed items are never re-created (`Left removed → ...`). `gt` never writes this file. Removing a rule file does not switch off a hook that enforces it (`enforcement: validated`); the install warns when that is the case — use `gt_settings.py` for that.

`/gt:gt-create`

Scaffolds a project. Gathers slug, title, tags, **domain**, sub-project parent, runbook, and **topology**; runs the script; then fills `idea.md` from what you actually said. `idea.md` is immutable afterwards — it is the traceable "why".

`/gt:gt-upgrade`

`install.sh` updates the **plugin**. This updates the **vault** — the migrations, the documents you also edit, and the Core rules a release added since your vault was seeded.

```
python3 $SCRIPTS/gt_upgrade.py status --vault "<vault>"      # what is pending, and why
python3 $SCRIPTS/gt_upgrade.py run --vault "<vault>" --dry-run  # rehearse; writes nothing
python3 $SCRIPTS/gt_upgrade.py run --vault "<vault>"          # apply, after a backup
```

Say what is pending before anything else, then rehearse. `--dry-run` writes no files, no backup and no stamp. It exists because on 2026-09-11 a rehearsal run by hand wrote the live vault instead of the copy.

The stamp is what makes any of this possible. A vault records the release its files came from in `Projects/golden-thread/.vault-version.json`. A vault with no stamp is treated as **old**, not current: every migration is offered, and each detects its own work, so a step that already happened is a no-op rather than a duplication.

SINCE	STEP	WHAT IT DOES
0.11.0	<code>log-spool</code>	<code>log.md</code> becomes generated from per-session spool files
0.11.0	<code>decisions-spool</code>	each project's <code>decisions.md</code> becomes generated
0.12.0	<code>core-rules</code>	adds the Core rules shipped since this vault was seeded
0.12.0	<code>doc-merge</code>	<code>PROTOCOL.md</code> and <code>CONVENTIONS.md</code> take the release's changes

The merge base lives in the vault, at `Projects/golden-thread/.templates/` — the exact template text the vault was last seeded or upgraded from. Only the current and previous release directories stay on disk, so a base kept in the plugin would vanish exactly when an old vault needed it. Without a base, a three-way merge degenerates into *overwrite* or *leave alone*, both wrong, so gt waits for a person instead.

`run` refuses a dirty git tree. The upgrade should be one `git checkout` away from undone, which it is not if it lands on top of someone's uncommitted work. `--allow-dirty` exists and is not the default answer. `run` backs the vault up to `~/claude/golden-thread/backups/` first, applies each step, stamps the vault, and writes a line to `log.md` through `gt_log.py`.

Exit `1` means a step needed an owner. Those lines print from `status` and `run` on every call until they are dealt with, and `install.sh` prints them too without ever acting on them:

WHAT YOU SEE	WHAT IT MEANS
<code>REFUSED: ... duplicate ADR number(s)</code>	two decisions share a number. Renumbering breaks every inbound reference, so the owner picks — never unasked
<code>CONFLICT</code> on a document	the merged text with markers is written beside it as <code><doc>.merge-conflict</code> ; the document itself is untouched
<code>needs a person: no merge base</code>	gt never recorded a base, so it cannot tell your edits from the release's changes. Review against the template, then <code>run --record-base <doc></code>
<code>needs a person: merge held</code>	a clean merge would remove lines you still have. Keep yours with <code>--record-base <doc></code> , or take the merge with <code>--accept-merge <doc></code>
<code>needs a person: conflict awaiting you</code>	a conflict gt already wrote for this exact base, template and document. It is not redone — no merge, no stamp, no rewritten conflict file — until something changes

A refusal stops **that step, not the upgrade**: everything else still runs, the refusal is reported, and re-running later picks up where it stopped. `--record-base` records the *shipped template* as the base, meaning "reviewed against this release", so later releases merge only their own changes, unattended — and only this command records a base; the installer never does.

It will not renumber anything, guess at a conflict, apply a line-removing merge unasked, redo a conflict it has already written, or overwrite a Core rule that already exists — new rules are added, existing ones left alone.

Writing to the shared files

`log.md`, `decisions.md` and `TASKS.md` are **generated**. They are the files every session writes and none owns, and in one working tree that means last-writer-wins with no conflict marker to warn you. So none of them is written directly.

```
T=<vault>/Projects/golden-thread/tools

# A log entry. Writes only this session's spool file, then re-renders log.md.
python3 $T/gt_log.py --vault <vault> add "2026-01-01 10:00 CST [work] my-project - what happened"

# An ADR. Reserve the number FIRST -- two sessions that both read "the highest is 5"
# will both write ADR-6. This takes the number in a single atomic step and prints it.
python3 $T/gt_adr.py --vault <vault> allocate my-project --title "The choice"
#   -> 7          (and creates the file that holds ADR-7; write the body into it)
python3 $T/gt_adr.py --vault <vault> merge my-project

# Who has spooled what, and any allocation left unfinished
python3 $T/gt_log.py status
python3 $T/gt_adr.py status my-project
```

Adopting this in an existing vault is one command per file. It freezes what is there as a baseline that sorts first, so no history is rewritten and no ADR is renumbered, and it refuses unless the merge reproduces the original byte for byte:

```
python3 $T/gt_log.py --vault <vault> migrate
python3 $T/gt_adr.py --vault <vault> migrate my-project
```

`migrate` refuses on a project whose ADR numbers already collide, because renumbering would invalidate every existing reference to them. `gt-lint` 's `adr-collision` reports those so you can decide. A deliberate `## ADR-6 amendment:` is not a collision and is left alone.

Movement events. `gt_events.py` records *what moved where* as structured JSON lines — the data `/gt-flow:gt-flow` draws. Same spool pattern as the log: one file per session under `spool/events/`, merged into `events.jsonl`. The tool and schema (v1) shipped in 0.13.0; since 0.15.0 operations record events as they happen:

- `gt_log.py add "<line>" --event <kind> --item <path> [--from --to --level-from --level-to --project]` spools the log line and one event in one command. `/gt:gt-promote` and `/gt:gt-refresh` record their moves this way; `/gt:gt-review`, `/gt:gt-work` and `/gt:gt-ingest` emit theirs with `gt_events.py emit`.
- `gt_adr.py allocate` emits an `adr` event for each number it reserves.
- `vault_init.py` emits `create`, `rename`, `merge` and `archive` when `create-project`, `rename-project`, `merge-project` and `archive-project` actually do something (never under `--dry-run`). `archive-project` is the only source of `archive`: `gt_closeout.py answer <slug> yes` records your decision to close, not the move, and emits no event.
- `gt_tasks.py` emits `task.open` / `task.done` for checkbox lines that changed — once the stream has been seeded with task events by a backfill.

An event can never fail the operation it describes: a failure is one line on stderr.

A vault older than the events has none. `backfill` rebuilds the history from git and `log.md` — Knowledge and global-memory additions, renames between levels, project renames, README checkbox changes — as `actor: backfill` events. Preview it first; a second run adds nothing. (A dry run on the author's vault found about 1,430 events.)

```
python3 $T/gt_log.py --vault <vault> add "2026-01-01 [graduate] quote_api → Knowledge" --event promote \
  --item Knowledge/Quote-API.md --from Projects/ats/memory/quote_api.md \
  --to Knowledge/Quote-API.md --level-from 2 --level-to 4 --project ats
python3 $T/gt_events.py --vault <vault> backfill --dry-run # what history would be added
python3 $T/gt_events.py --vault <vault> validate # report bad lines, exit 1 if any
python3 $T/gt_events.py --vault <vault> list --kind promote --json
```

Every event is checked before it is written: vault-relative paths only, a closed list of kinds, levels 1–5, a note of at most 120 characters, no unknown keys.

Hand-editing a generated file is still the wrong place for it, but it is no longer lost (0.15.0): a line typed into `log.md` is captured into `spool/log/hand-edits-<time>.md` at the next merge and announced — a line starting with a date sorts by it, an undated one lands after the baseline — and `gt_adr.py merge` refuses to render over a hand-written line in `decisions.md`, naming it, until you move it into an ADR. `gt-lint` reports the file as `generated-hand-edited`.

Both commands sit on one shared primitive, `gt_spool.py`: per-session spool files plus a merge whose ordering uses nothing machine-specific, so two machines rendering the same spools produce byte-identical output. Bytes that are not valid UTF-8 (an old latin-1 `é` in `log.md` or `decisions.md`) pass through `migrate` and `merge` exactly as they were (0.15.0). You do not call it directly; it is documented here because a merge that ordered by mtime or directory order would make the *generated* file conflict in git, which is worse than the problem being solved.

Daily work

`/gt:gt-open <slug>`

Loads a project and stops. It is **read-only** — nothing is written during loading.

Reads, in order: `CONVENTIONS.md` and `PROTOCOL.md` (once per session), the project `README.md` — frontmatter first, since `domain / stage / topology / tags` is the fastest read of where the project stands — then `source.md` → `idea.md` → `research.md` → `decisions.md` → `design.md` → `spec.md` → `runbook.md`, the linked fleet page, and finally `memory/MEMORY.md` — the index only. Individual memory files load on demand.

Then the project's waiting handoffs (0.17.2) — it runs `gt_surface.py handoffs --project <slug>`, which prints one line per handoff of this project that has not been handled (or whose deferral date has come), and nothing when `handoff_surface` is `manual`. Each is named in the summary with an offer of [/gt:gt-handoff-handle](#); its body is not read unless you want to handle it now, because being told must cost no project context. A handoff exists because the last session could *not* capture everything, so its uncaptured items are the ones no other file holds. Until 0.17.2 this skill never looked, and a handoff reached the next session only if someone remembered it existed.

source.md leads the project docs deliberately. It says which box serves which role in which environment. Acting before reading it is how you edit the wrong host, or overwrite a file that exists on three machines.

research.md over ~200 lines is read by its headings, plus the entries relevant to the work at hand and the most recent few. It is append-only and grows without bound; reading it whole crowds out the context the actual work needs.

Sub-projects load only when you name one, or the status board shows it as the active item.

Announces the `review-queue.md` count once, then summarises stage, **topology and hosts**, next action, blockers, and what memory exists but is unloaded — the host list so you can correct a stale entry *before* work starts rather than after.

/gt:gt-route

The middle of a session. `gt-open` runs before the session knows what it is; `gt-work` runs after it is over. Between them sits the part where the work actually changes shape, and nothing served it.

Run it whenever the ground has shifted, or whenever you cannot name what you are doing. It reads only the conversation already in context — never project files, because reading `research.md` to decide where a note goes costs more than the note — and answers four things:

1. **What this has become.** Restated in a sentence, with the drift named explicitly if the session no longer resembles what it opened with. This is most of the value: the drift is usually felt before it is articulated.
2. **Where the output belongs** — the [Where does this fact go?](#) table, applied to the material actually in hand, naming more than one destination when more than one is true.
3. **Whether the work is in the right place** — project loaded, harness (a real PTY belongs in a terminal; vault context belongs in Claudian), and whether the model matches the difficulty. It reports mismatches; it changes nothing.
4. **One next action.** A skill to invoke or a file to append to — not a menu.

It **writes nothing**, with a single exception: one `INBOX.md` checkbox line, which is what `INBOX.md` is for. Everything else is handed to the skill that owns the destination, which does the writing under its own claim.

"Nothing, keep going" is a valid answer and the skill is built to give it. A check that always finds work is a check nobody runs twice.

Use it when a session opened as one thing and became another; when a finding might belong to a project other than the one loaded; when you are about to write something down and are not sure which file; or when you simply feel lost. Being wrong costs one re-run.

/gt:gt-work

End of session. The step people skip, and skipping it is what makes the vault decay.

GOES TO	WHAT BELONGS THERE
<code>research.md</code>	Dated findings and gotchas. New discoveries only.
<code>decisions.md</code>	Numbered ADR: decision, context, rejected alternatives. Stable choices only.
<code>design.md</code>	Rewritten in place if architecture changed. Describes <i>now</i> .
<code>memory/*.md</code>	Session state, updated in place.
<code>spec.md</code>	Created when design is complete enough to hand off.

Memory files carry `level` and `enforcement` in frontmatter to make a rule's durability explicit:

```
metadata:
  type: core | feedback | user | reference
  level: core | context | generic      # default: generic
  enforcement: validated | reminder    # required only for level: core
```

Default to `generic`. Do not set `level: core` here — Core is a promotion, and it requires wiring an enforcement hook, which is `gt-promote`'s job.

After writing, `gt-work` actively asks whether any finding applies beyond this project and should go to `Knowledge/`, `global-memory/`, or a new project.

Also updates the `stage:`, `topology:`, and `domain:` properties in `README.md` when the project's reality changes.

Not in `decisions.md`: anything you might change next session. An ADR you reverse next week teaches the vault to lie to you.

It offers a handoff for what did not reach a file (0.17.1). Classifying findings tells `gt-work` not only what to write but what it is *not* writing — a decision that needs you, something mid-flight, a question raised and unanswered. Those live in the conversation, and the conversation is about to end. So it names them and asks whether to write a [handoff](#); on yes it runs `gt_handoff.py` and writes the narrative itself, because the script deliberately will not.

On yes it also raises the work, one task per unresolved item in the project's `## Tasks`, each naming its own item and citing the handoff:

```
- [ ] Decide whether the sweep should block on cannot-run - context in handoff-2026-09-28.md [p:: 1] [waiting:: user] [since:
```

A handoff nobody is told to read is write-only, so the task is the mechanism and the file is only the context. `p:: 1`, never `p:: 0` — there is no `p:: 0`; `0` exists at project level only, where it is defined as a tier nothing occupies permanently. `since::` is what makes it escalate: the vault's stale-P1 escalation rule raises the project a level once the task is seven days old, so an unread handoff climbs on its own and a read one costs nothing. One task per item, never a single "review the handoff" line — that competes with real work while saying nothing, and hides every item after the first.

The question is asked only when the list is non-empty, and that restraint is the feature. An offer made every session is one that is always declined — `gt_closeout` demonstrated exactly that by asking the same question every session until the answer stopped being read. Nothing uncaptured, no question, one line saying so. And a `no` is final: nobody writes a handoff on your behalf after you have said those items are not worth a file.

The next session is told, without anyone asking (0.17.2). `gt_surface.py` runs at `SessionStart` and names every handoff not yet handled, every session, with its count of open items. A handoff counts as **handled** once every task citing it is checked off — so **the task must name the handoff's filename**, as in the line above. That filename is the join; a task that does not name it is invisible to both the count and the automatic close.

`/gt:gt-handoff`

The session that *designs* something is rarely the session that builds it. This writes down what the next one needs — and marks what it must not assume.

```
python3 $SCRIPTS/gt_handoff.py --vault "<vault>" --project <slug> [--repo <code-path>]
```

Writes `Projects/<slug>/handoff/<date>-handoff.md`. `--json` returns the facts without writing a file; `--force` is required to replace an existing handoff, because a handoff is someone's record of a session and overwriting one silently loses it. Since

0.17.2 it opens with frontmatter — `type: handoff, project, status: open, created` — and `status: open` is what keeps it in front of people until someone deals with it.

The script does half the job on purpose. It gathers **facts** — the project's stated goal, its open tasks, its recent decisions, the observed state of the code repository, what is uncommitted, what the last commits were — each carrying where it came from, so the next session can go and check it. It deliberately does **not** write the design narrative. A script that invents "what we decided and why" produces a document that *reads* finished and is not, and the next session inherits false confidence instead of no confidence.

That section is yours, and four things belong in it:

- **What was decided, and what was rejected.** The rejected options are what the next session will otherwise re-propose and rediscover the hard way. This is the most expensive thing to lose.
- **What is built versus designed.** Be blunt: "designed, not written" and "written, not tested" are different states, and a file listing cannot tell them apart.
- **What to do first**, and what would make it wrong.
- **What is uncertain.** An open question written down is worth more than a confident guess.

Verification labels are not decoration. Every fact is labelled `verified`, `unverified` or `unknown`, per Core rule 10, and the script never claims verification: it says what it observed and marks the rest. If the handoff says the tests pass, say **who ran them, when, and which ones** — "the tests pass" becomes folklore the moment it is written down in something formal-looking. Anything you did not personally observe this session is `unverified`.

The document opens with a checklist of questions the script cannot answer — whether the tests pass, whether the decisions were overtaken later in the session, whether anything agreed verbally never reached a file. Answer them in the document, or say plainly that they are unanswered. The script says so itself when it finishes: the output is **not** a finished handoff, and one left with its placeholder intact should be reported as incomplete rather than read as a design.

Log the run in `log.md` with `work`, and link the handoff from the project's `README.md` so the next session finds it without being told it exists.

Exit codes: `0` written · `2` usage (`--project` takes a slug, never a path) · `3` the project could not be read, the target exists without `--force`, or the write would land outside the vault.

`/gt:gt-query`

Reads `index.md`, follows wikilinks 2–3 hops into `Knowledge/`, falls back to grep, then project memory. Be specific. If a result is `status: stale`, verify it.

`/gt:gt-ingest`

Bulk-imports an existing project. Copies — never moves or deletes.

The trap: if the notes cross-reference each other by `[[wikilink]]`, renaming or merging files destroys that graph, because Obsidian resolves links by filename. Measure first:

```
grep -roh '\[[^\]]*\]' --include='*.md' . | wc -l
```

Prefer keeping filenames. If you must reprint, match the *whole* link — `[[name]]` and `[[name|alias]]` — never a substring, or `[[foo]]` will corrupt `[[foo_bar]]`. Never reprint links inside `Sources/`.

/gt:gt-review

Empties `INBOX.md` : reads every unchecked line (and daily notes, if configured), asks where each belongs, files it as a task or project, and checks the inbox line off with a pointer. Regenerates the rollup at the end.

Tasks and handoffs

Two kinds of "someone has to come back to this": a **task** (one line in a project README's `## Tasks`) and a **handoff** (a file a session wrote because it could not capture everything). Since 0.17.2 both work the same way, and five rules hold for both:

- **Surfacing is the alarm; the task or handoff is the record.** Session start *tells* you what is waiting; the README line or the handoff file is what gets changed when you deal with it.
- **Three verbs: create, see, handle.** `/gt:gt-task` creates; `/gt:gt-task-list` and `/gt:gt-handoff-list` only show; `/gt:gt-task-handle` and `/gt:gt-handoff-handle` work through them. Seeing is kept separate from handling on purpose.
- **Nothing is deleted.** Done and dropped tasks are checked off with what settled them; a handled handoff keeps its file and gains a status line. Dropping needs a reason.
- **A deferral needs a date.** It hides the item until then and brings it back on its own. "Later, some time" is a drop, said out loud.
- **Being told costs no context.** What is shown at start is one line per item or a count — never a handoff's body, never a project opened.

/gt:gt-task

Create a task the way a developer drops a TODO into code: say it in your own words, and the skill writes one well-formed line into the project README's `## Tasks` through the vault tool `gt_task.py` , so it ranks in `TASKS.md` exactly as a hand-written one would.

```
python3 <vault>/Projects/golden-thread/tools/gt_task.py add "<task>" --vault "<vault>" \  
  (--project <slug> | --inbox) [--ref "<[[Page]] or vault path>"] [--p N] \  
  [--waiting user|agent|external|parked] [--due YYYY-MM-DD] [--dry-run]
```

The skill decides what it can: the project this session is in (or asks once, offering `--inbox` for `/gt:gt-review` to route later), `p:: 2` unless you say urgent (`1`) or someday (`3`) — never `p:: 0` , which does not exist for tasks — and `waiting:: user` unless the work is plainly the assistant's or someone else's. `[since::]` is stamped automatically.

A task can be tied to what it is about with `[ref:: ...]` — a `[[wiki page]]` or a vault path (`Sources/...` , `Projects/x/design.md`). The ref **must resolve**; the tool refuses one that points at nothing and says what it looked for.

One store, one parser. Tasks stay in the README; `gt_task.py` is a writer and a reader for that section, not a second database, and it parses with `gt_tasks.py` 's own rules. Hand-written tasks are unchanged and work with every command here. **Core rule 1 holds:** the tool refuses to write a README another live session has claimed.

/gt:gt-task-list

Read-only: open tasks, filtered, without loading any project's context. Most urgent first.

```
python3 <vault>/Projects/golden-thread/tools/gt_task.py list --vault "<vault>" [FILTER ...] [--json]
```

A task must match every filter given:

FILTER	SHOWS
<slug>	one project (sub-project as <code>parent/child</code>), or <code>inbox</code>
p1 p2 p3	that priority
mine	<code>waiting:: user</code>
overdue	<code>due::</code> before today
stale	<code>p:: 1</code> open for more than 7 days
deferred	only tasks deferred to a later date — hidden otherwise
<code>ref:<text></code>	tasks whose <code>ref::</code> contains <code><text></code>

Each line starts with an ID, `slug:LINE:HASH` — the README line number and six hex of the line's text. Shelved tasks (`p:: 7` +) do not show unless a filter asks for them.

/gt:gt-task-handle

The way to clear a backlog: choose a set with the same filters (`/gt:gt-task-handle stale , ... my-project p1`), then take the tasks one at a time. Context is loaded only when a decision needs it — the file a `ref::` points at, or one named file, never a full `/gt:gt-open` .

DECISION	COMMAND
done	<code>gt_task.py done <ID> --vault V [--reason "<what settled it>"]</code>
drop	<code>gt_task.py drop <ID> --vault V --reason "<why it no longer matters>"</code> — reason required
defer	<code>gt_task.py defer <ID> --vault V --until YYYY-MM-DD --reason "<why then>"</code> — future date and reason required
keep	nothing; move on

Done and drop check the box and append `— done <stamp>: ...` or `— dropped <stamp>: ...` to the line. Defer adds `[defer:: YYYY-MM-DD]` : the task is hidden from `list` , from the `TASKS.md` ranking and from escalation until that date, then comes back on its own. An ID is refused if its line has changed since it was listed — an ID from a stale list can never close the wrong task; list again and use the new one. At the end the skill regenerates `TASKS.md` .

A dropped task reads as **closed** to everything downstream — `gt_events` , `gt_daily` — because its box is checked; the line itself records `dropped: <reason>` .

/gt:gt-handoff-list

Read-only: the handoffs still waiting — open, or whose deferral date has come — one line each (path, project, age, open items), oldest first, across every project or one. `--all` shows every handoff with its state. **No handoff body is read**: listing must cost no project context, which is why listing and handling are separate skills.

```
python3 $SCRIPTS/gt_handoff_status.py list --vault "<vault>" [--project <slug>] [--all] [--json]
```

/gt:gt-handoff-handle

Where a waiting handoff gets dealt with. It lists what is waiting as a menu and **loads nothing until you pick one** — then only that handoff and the `## Tasks` lines citing its filename, not a full `/gt:gt-open`. Each open item is settled one of three ways:

- **Done** — the task is checked off with a note of what settled it; a decision goes to `decisions.md` or `research.md` through the usual `/gt:gt-work` path.
- **Keep** — the task stays open. It is already `p:: 1` with a `since::` date, so it escalates the project on its own after a week.
- **Drop** — checked off with the reason. Dropping is a decision, recorded, never a deletion.

Then the handoff's status is set:

```
python3 $SCRIPTS/gt_handoff_status.py mark <handoff> --vault "<vault>" --status handled --reason "<one line>"
python3 $SCRIPTS/gt_handoff_status.py mark <handoff> --vault "<vault>" --status deferred --until YYYY-MM-DD --reason "<why>"
```

Four states, and what decides each:

STATE	MEANING
open	written by <code>gt_handoff.py</code> , or re-opened. Shown until handled or deferred
deferred	put off to a date (<code>--until</code> , which must be in the future). Hidden until then; on that date it is open again. There is no deferral without a date — "later, some time" is a decision to drop it, so it is handled with that reason, said out loud
handled	marked so by a person, or every task citing the handoff's filename is checked off — the tasks are the record, so closing the last one needs no second step
history	written before 0.17.2 (no <code>status:</code>), over a week old, with no open task citing it — so an upgrade does not resurface every old handoff in the vault

A handoff with no `status:` counts as open while it is under a week old or any open task cites it. `mark` appends a line to the handoff's status log and refuses a file outside the vault. A handoff is never deleted — it is someone's record of a session; its status says it is dealt with.

What you see at session start

`gt_surface.py` runs at `SessionStart` and puts four things in front of the session. A clean start says `nothing waiting` in one line, because silence reads the same as a hook that never ran.

WHAT	HOW OFTEN	SETTING
MUST DO — rows of <code>deadlines.md</code> overdue (🔴 OVERDUE Nd) or due within 14 days (🟡); later rows counted, not listed	every session, recomputed live	<code>surface</code>
Handoffs not yet dealt with — one line each: path, project, age, open items; never the body	every session until handled or deferred to a date	<code>handoff_surface : any</code> (default) · <code>project</code> (only when <code>/gt:gt-open</code> opens that project) · <code>manual</code> (only in <code>/gt:gt-handoff-handle</code>)
Tasks — one line: how many <code>p: 1</code> tasks wait on you, how many overdue, how many open over a week, and the commands to list and work them	every session while any wait	<code>task_surface (on · off)</code>
State files <code>gt_state.py</code> wrote before a compaction	once each; right after a compaction the newest one's content is handed to the model in full	<code>surface</code>

Why handoffs repeat. The first 0.17.2 cut showed a handoff once and went quiet, on the `gt_closeout` lesson that a question asked every session trains people to scroll past it. The owner ruled the same day that the opposite failure is worse: a handoff scrolled past on a busy morning is as good as never written. So an unhandled handoff comes back every session, and the cost is kept to one line. `any` is the default because under `project` a handoff waits until someone happens to open its project.

The task line is a count, never the tasks, and a deferred task is not counted until its date. It is counted by the vault's own `gt_task.py` , so a vault not yet refreshed to 0.17.2 (run `/gt:gt-upgrade`) shows no task line rather than a guess.

`deadlines.md` format

One vault-root file with one table, one row per outstanding dated item — a credential rotation, a decision with a deadline, a renewal:

```
| item | category | due | see |
|---|---|---|---|
| Rotate the relay credentials | rotation | 2026-10-03 | [[pending-rotations]] |
```

`due` is `YYYY-MM-DD` . Ages and countdowns are derived from the date at run time, never stored, so the block counts itself down and disappears when the last row is deleted. Adding a category is a row, not code. The detail — a rotation's ordered steps, a decision's comparison — stays in the file `see` points at, because a rotation's danger is its *sequence* and a one-line alarm cannot carry that. A row that does not parse is skipped **and counted**; the rest still report. A label shaped like a credential is **withheld**, never printed, because this output reaches both your terminal and the model's context. `gt_surface.py must-do` prints the block on its own.

Context management

`/gt-farm:gt-farm`

Module `farm` (plugin `gt-farm`). Off for a fresh install — `bash install.sh --with farm` adds it. A machine upgrading from a `gt` that shipped `/gt:gt-farm` keeps it on.

Routes work out of this context to an external AI service when work is bulk, mechanical, or genuinely benefits from a non-Claude second opinion.

The unit is a **work packet**: self-contained, with a strict return contract (**FINDINGS** + **GAPS** blocks). The transport is swappable — the user pastes it into a web UI today; a script sends it to an API later. The packet is identical either way.

All four gates must pass before a task leaves. Any failure means the work stays here.

GATE	QUESTION
Stateless	Answerable with no vault or repo state?
Self-contained	Fits in a paragraph plus a list of URLs?
Checkable	Verifiable without redoing the work?
Releasable	Every input safe to hand a third party?

Releasable is default-deny. Never include: account identifiers, balances, hostnames, internal IPs, credential locations, non-public source code, or personal identifiers.

Key rules: - One packet, one task. A packet asking three questions returns three half-answers. - Supply a vocabulary block with exact model IDs, product names, and version numbers read from the system — never from memory. - Results come back as **unverified** . Promote a finding only after opening its **source:** URL and confirming the claim. - **GAPS** is mandatory and **GAPS: None** must be justified, not asserted. - Cite the specific page carrying the claim, never a homepage.

Packets are saved to `<packet dir>/<YYYY-MM-DD>-<slug>.md` . The packet dir is `farm_packet_dir` in `~/.claude/vault-config.json` if set, else `Projects/<current project>/packets/` . A relative value is taken from the vault root and a literal `<project>` in it becomes the current project's slug; an absolute value is used as is. The skill names no particular AI service: which services you have is your configuration (`farm_services` in `vault-config.json` names a table of them). Copying a packet uses whichever clipboard tool the machine has — `pbcopy` , `wl-copy` , `xclip` or `clip.exe` — and with none, you are given the packet file's path instead.

Knowledge management

`/gt:gt-promote`

The judgement call: **has a second project proved this general?**

FROM → TO	WHEN
memory/ → research / decisions	The finding stopped changing session to session
project → Knowledge/	A second project hit the same truth
Knowledge/ → global-memory/	Needed in <i>every</i> session, <i>every</i> project
anywhere → new project	The idea doesn't belong where it is

Nothing is deleted. Retiring sets **status: stale** and records what superseded it.

Promoting a rule to Core (**level: core**) also wires the enforcement hook — `gt-promote` handles this; `gt-work` does not.

gt_demote.py — the same ladder, downwards

```
python3 $SCRIPTS/gt_demote.py --vault "<vault>" --file "<path relative to the vault>" \
  [--to knowledge|project-memory] [--project <slug>]
python3 $SCRIPTS/gt_demote.py --vault "<vault>" --file "<path>" --apply
```

The two ladders measure different things. `/gt:gt-promote` moves knowledge *up* by how settled it is. This moves it *down* by how often it is paid for:

LEVEL	READ	COST
global-memory/	every session of every project	most expensive
Projects/<slug>/memory/	every session of one project	
Knowledge/<page>.md	when someone asks for it	cheapest

A fact can be perfectly true, perfectly settled, and still be in the wrong place — because it is loaded into every session of every project and needed in one. Demotion is the answer to that, and it is not a demotion in the sense of being wrong.

Nothing is deleted, and nothing moves without `--apply`. Without it you get the move it would make and the reasons it would refuse; exit `1` is a refusal with nothing written.

/gt:gt-refresh

Checks `Sources/` for upstream changes. Supersedes with a new immutable file carrying `supersedes:` — never edits the old one.

Verification

/gt:gt-validate

Verify a claim by **re-deriving** it, not by reviewing it. Use before a finding is recorded as fact, before a production change, or whenever a number matters.

The one rule: the validator never receives the reasoning that produced the claim. Give it the reasoning and it grades the argument — inheriting the same blind spot. Give it only the claim, the rules and the artifact, and it has to go back to primary sources. That is the only thing that catches a wrong premise.

Reduce the work to a **single falsifiable assertion**. "The analysis is sound" is not validatable; "widening NQ's target to 1.25 improves risk-adjusted return" is. Validate several claims **separately** — a bundled claim returns a bundled verdict, which hides which part failed.

Loads the project's `validation-rules.md`, and the parent's too, since packs are inherited. Those are standing invariants the validator enforces whether or not the request mentions them — because *a requester who has already made a domain error will not think to ask the validator to check for it*. On conflict, the pack wins.

CLASS	USE WHEN THE RISK IS
<code>empirical</code>	A number, measurement or result could be wrong
<code>vantage</code>	The measuring position may not be able to observe the answer
<code>rule-compliance</code>	Work must satisfy <code>core-rules/</code> or <code>CONVENTIONS.md</code>
<code>code</code>	Code may not do what its name, comment or docs claim

Pick every class whose risk is present. **Do not substitute one generic reviewer** — that produces agreement, not verification.

The packet is exactly three fields — `claim`, `rules`, `artifact` — then re-read it and strip your conclusions, your numbers, your confidence, and any adjective implying the expected answer. "Confirm that X" becomes "determine whether X". Withholding *reasoning* is isolation; withholding access is just a broken validator, so keep the hostnames, API shapes and filters it needs to reach the artifact alone.

Run the validators **independently**, and read their derivation *before* re-reading your own — reading yours first turns the exercise back into review.

Three verdicts: **confirmed**, **refuted** (quantify the divergence), and **cannot-verify** (name what was missing).

Append the verdict to `research.md`. A refutation supersedes the original finding.

cannot-verify is never a pass. A validator that could not check something and stayed quiet manufactures false assurance — worse than no validator at all.

You are the worst possible author of this packet, because you already know the answer and your framing leaks. Treat building it as an adversarial exercise against yourself. A validator disagreeing is a *result*, not a failure.

`/gt:gt-validation`

New in 0.16.1. Records what a validation **established** about one file, stamped with that file's **content hash**, so the claim expires the moment the file changes.

It records a result; it does not produce one. Run `/gt:gt-validate` first — this is the step after it.

```
python3 $SCRIPTS/gt_validation.py record --file <path> --verdict holds \
    --checked "one property that was verified" --gap "one thing that could NOT be determined" \
    --by "who or what ran it"
python3 $SCRIPTS/gt_validation.py check --file <path>      # 0 covered · 1 stale · 2 never validated
python3 $SCRIPTS/gt_validation.py show --file <path>      # the definition, with its gaps
python3 $SCRIPTS/gt_validation.py list                    # everything, and what has gone stale
```

`--verdict` is `holds`, `broken` or `cannot-verify`. `--checked` and `--gap` are repeatable — one flag per property, not one flag holding a paragraph.

Why it exists. Every serious defect found in 0.16.0 was a *claim that outlived its implementation*: a manifest row shape that stopped matching, a `lint` check listed as running while wired to nothing, an aggregator counting installed rather than declared members. Each was true when written, and nothing tied the claim to the code's current state, so nothing could notice when it stopped being true. A docstring is a claim; this is a claim with an expiry date attached to the bytes it describes.

The hash, not the clock. A test receipt uses time, because a run covers a whole tree and the clock is the only thing the editor and the runner agree on. A validation is about *one file's* behaviour, so the file's bytes are the better anchor: a hash cannot drift with a clock,

survives a copy between machines, and is exact — any edit at all invalidates the claim it was made about.

`--gap` is not optional in spirit. A receipt that flattens to "validated ✓" manufactures exactly the assurance this system keeps having to dig back out. Record what was *not* established, in the artefact, permanently. A validation that genuinely left nothing undetermined is unusual; if that is really so, say so with `--gap none` rather than by omission — `record` prints a note when no gap was given, and `show` renders an empty gap list as *(nothing recorded, which is itself unverified)*.

check SAYS	EXIT	MEANS
covered	0	the file is unchanged since it was validated, and the verdict was holds
STALE	1	the file changed after it was validated — what was established no longer describes it
MISSING	1	there is a receipt, but the file cannot be read now
VALIDATED-BROKEN / -CANNOT-VERIFY	1	current, and the verdict was not holds — said first, because "covered" read as reassurance
NEVER VALIDATED	2	unknown, not clean

Exit 2 means unknown, not clean. A file nobody validated is not a file that passed, and `check` keeps those two states apart deliberately — the same rule as "a member that could not run is not a pass".

Rules, each one learned from a failure:

- **Never record a receipt for a validation you did not actually run.** A recorded assumption is worse than no record, because it looks like evidence.
- **Never quietly re-record to clear a stale one.** Stale means the file changed after it was validated; the answer is to validate it again, not to re-stamp it.
- **Say the state when you cite a definition** — `covered` , `stale` or `never` . Citing a stale receipt as current is the failure this exists to prevent.

The ledger lives in the repo, at `dev/validations.jsonl` — plain JSONL a reviewer reads in a diff, appended to, never rewritten. That is the opposite of a test receipt, and deliberately: a test receipt attests that a tree passed *here*, so it is machine-local; a validation attests to what the code **does**, which is a property of the code and travels with it. Off a checkout it falls back to `~/.claude/golden-thread/validations.jsonl` . A row counts as evidence only if it has the shape of evidence — a real path, a 64-hex sha256, a known verdict, a list of checks and a timestamp — and the newest receipt for a file wins *by recorded time*, not by file order.

Maintenance

/gt:gt-lint

CHECK	CATCHES
index-gap	Knowledge page not in <code>index.md</code> — <code>gt-query</code> won't find it
broken-link	<code>[[wikilink]]</code> resolving to nothing
orphan	Knowledge page linked from nowhere
memory-unlisted	<code>memory/*.md</code> not in <code>MEMORY.md</code> — Claude will never load it
global-gap	<code>global-memory/*.md</code> not in its index
memory-bloat	<code>global-memory</code> file over 30 lines
global-scope-leak	Project-specific content in <code>global-memory/</code>
superseded-cited	Knowledge page citing a superseded source
stale	Knowledge page marked <code>status: stale</code>
source-todo	<code>source.md</code> with no topology or deployment targets
frontmatter	Project README missing properties, or slug $\neq$ folder
core-misplaced	A rule declares <code>level: core</code> but lives outside <code>core-rules/</code>
core-no-enforcement	<code>level: core</code> with no <code>enforcement</code> field declared
core-unenforced	The declared enforcement hook is not actually wired
adr-collision	Two ADRs share a number in one project
generated-hand-edited	<code>log.md</code> or a <code>decisions.md</code> edited by hand instead of through the spool
attribution-unwired	<code>gt_edits.py</code> is in the vault but the git attribution hooks never run — <code>core.hooksPath</code> unset, or <code>prepare-commit-msg</code> / <code>post-commit</code> missing or not executable
project-missing	A link points at a project folder that no longer exists
runbook-duplicate	A line duplicated across two or more projects' <code>runbook.md</code> — <code>--runbooks</code> only, and the detection step of <code>/gt:gt-runbook-lint</code>

`gt_lint.py` emits **eighteen** check kinds, and every one of them runs on an ordinary pass, each wired to a function the run actually calls. `runbook-duplicate` is not one of the eighteen: it is a separate read-only mode, `gt_lint.py --runbooks`, which runs nothing else and reports under its own record shape — the detection step of `/gt:gt-runbook-lint`. Fewer than two runbooks prints "nothing to compare" and exits 0. `core-unenforced` is the critical one — it is the machine-checkable form of "rule stored but never asserted". Treat it as a real defect. **Do not suppress it** — suppressing `core-unenforced` recreates the original bug with a paper trail saying it was fine. Fix: `vault_init.py install-core-rules --vault <vault>`.

Declines go in `lint-declines.md` as `suppress: with the reason`.

Triage findings into three piles: *you broke it* (fix now), *already broken* (record in `review-queue.md` — don't guess at a target), *false positive* (suppress with reason).

/gt:gt-doctor

The whole install in one report: the checks that already existed in five scripts and a SessionStart message, asked for on purpose instead of scrolling past at login.

```
python3 $SCRIPTS/gt_doctor.py          # every check, human-readable
python3 $SCRIPTS/gt_doctor.py --only wiring # one check (repeatable)
python3 $SCRIPTS/gt_doctor.py --json    # for another tool to read
python3 $SCRIPTS/gt_doctor.py --fix     # re-wire hooks, and nothing else
```

Read the first line out loud. It names the release every other answer is relative to. A clean report from a check pinned to the wrong release reads exactly like a healthy install — that is how 0.9.4 sat uninstalled on 2026-08-30 beside a component check reporting clean against 0.6.0. Say the version, then the findings.

Twelve checks, each answering a different question:

CHECK	QUESTION
version	is the newest release the one installed?
components	do the installed files match that release?
wiring	is every hook the release declares actually in <code>settings.json</code> ?
core-rules	does each rule that claims enforcement have its own mechanism wired — the right script, at the installed path, present on disk?
modules	which modules are on or off, does each admit this gt, and is every ON module's plugin installed and enabled? (read-only)
vault	is the vault reachable, what release is it stamped at, and does <code>gt_upgrade</code> have anything pending for it?
schedule	has every installed scheduled job run, and did it last exit normally? (0.17.2)
workers	are background processes running that nobody declared?
push	do this machine's commits exist anywhere else?
gt-src	does the publish destination still match what was published? Since 0.17.3 every file is checked against gt-src's <code>SHA256SUMS</code> , and files changed since publish, missing, or not written by the publisher are each named (only where <code>\$GT_SRC</code> or <code>gt_src</code> in <code>vault-config.json</code> names one)
lint	what does the vault linter say, in one line?
astgrep	is the optional structural matcher here, and new enough? Absent is fine — its rules are SKIPPED, not silently passed

vault asks **gt_upgrade** itself (0.17.2). It used to hand-code the two 0.11.0 migrations, so it missed every migration since, never said what release the vault was stamped at, and called a project born in the new decisions format unmigrated. It now prints **stamped X, release Y** and lists whatever `/gt:gt-upgrade` would run, so the doctor and the upgrade cannot disagree. A stamp behind the release with nothing pending is a WARN, not a broken vault.

schedule reads **launchd's** own last exit for each job `gt_schedule.py` installed, judged by the scheduler's `BENIGN_EXITS` table rather than a second copy of it. A job never installed is a choice, not a finding; no installed job, or not macOS, is SKIPPED — never clean. It exists because a weekly lint failed on three Mondays running and nothing surfaced it.

EXIT	MEANING
0	all clear
1	needs attention, or broken
2	a check could not run — not the same as clean

Exit 2 is the one that matters: "could not check" reported as "clean" is the failure this whole tier exists to prevent. Say which check could not run, and why.

Every row that is not `ok` carries a `fix:` line. `--fix` repairs wiring only — it re-registers hooks that are declared and missing from `settings.json`, adding entries and removing none, so no configuration of yours can be lost. A pending migration, an unpushed commit and a stray worker each need a person to decide; offer the safe fixes, run none unasked.

/gt:gt-optimize

Memory is read into every session. A duplicated fact is paid for on every turn, in every project, forever — this finds that waste. It is not `/gt:gt-lint`: lint asks *is the vault structurally correct*, this asks *is the vault paying for words it does not need*. Different question, run both.

```
python3 $SCRIPTS/gt_optimize.py --vault "<vault>" [--project SLUG] [--json]
```

Reporting **never writes a byte**. Findings come in two lists: *mechanical* (decidable from the text — a dead index row, a run of blank lines) and *judgement* (reported, never applied). The judgement ones were always the valuable ones and they need a reader:

- `duplicate-fact` — the same sentence in two context-loaded files. Decide which one owns it and link to it from the other; usually the more specific scope wins.
- `memory-bloat` — a `global-memory/` file over the budget of 30 non-blank lines. Move the detail to `Knowledge/`, leave the fact. The finding prints the `--demote` command that acts on it.
- `relative-date` — "last week" in a file injected into a session months later. Only flagged in silently-loaded files, where it genuinely misleads.
- `duplicate-line` — check whether it is *structure* before removing it. Measured against a real vault, the hits were an ADR's required — ****Rejected alternatives****: label and a line shared by two Dataview queries. In prose a repeated line is waste; in a document built from repeated structure it *is* the structure, and no script can tell which it is looking at.

Only files loaded into a session are examined — `global-memory/`, `core-rules/` (read-only), a project's `memory/` and `CLAUDE.md`. Generated files (`TASKS.md`, `log.md`, `review-queue.md`, `events.jsonl`, `MANIFEST.json`) are skipped: editing one is pointless because the next run puts it back, and reporting one buries the findings that matter. `--project` takes a **slug, not a path**; a path, or a slug resolving outside the vault, is a usage error.

`--demote` : the only thing it writes (0.16.1)

There is no removing `--apply`. There was one, for about an hour on 2026-09-16, restricted to a "SAFE" class said to be decidable from the text alone. An independent validation took it apart: `dead-index-row` deleted seven live rows out of eight in the fixture, blank-run collapsing rewrote the inside of code fences, findings computed from one read were applied to another by line index, CRLF files were rewritten to LF and mode `0600` became `0644`, and `--project` was joined to the vault unvalidated so `../..elsewhere` edited files outside it entirely. It was **removed rather than repaired**: the SAFE class was a claim about the world, and the world kept producing exceptions. `--apply` without `--demote` is now a usage error.

Demotion is the shape that *can* be repaired, because **the ladder here is about cost, not maturity**. `/gt:gt-promote` moves knowledge up by how settled it is; this moves it down by how often it is paid for:

<code>global-memory/</code>	read in EVERY session of every project	most expensive
<code>Projects/<slug>/memory/</code>	read in every session of one project	
<code>Knowledge/<page>.md</code>	read when someone asks for it	cheapest

A fact in `global-memory/` that only one project needs is charged to every session forever. Moving it does not make it less true — it makes it cost what it is worth.

```
python3 $SCRIPTS/gt_optimize.py --vault "<vault>" --demote global-memory/thing.md
python3 $SCRIPTS/gt_optimize.py --vault "<vault>" --demote global-memory/thing.md --apply
```

Without `--apply` it previews and moves nothing. `--to knowledge|project-memory` overrides the default of one tier cheaper; `--project <slug>` is required to land in a project's memory.

The order is the safety property, and it is the entire reason this is a separate tool rather than another `--apply` flag:

1. **write** the destination, carrying provenance and a link back,
2. **verify** it by re-reading it from disk and comparing bytes — then re-read *both* sides again immediately before touching the source, because the invariant is not "it was there" but "it is there now",
3. **only then** replace the source with a pointer to where it went.

A failure at any step leaves **duplication** — the same content in two places, which a reader can see and resolve. The deleting version failed by removal, which no diff brings back.

It refuses rather than guesses. `core-rules/` and `Sources/` are **never** moved — compared casefolded and NFC-normalised on the resolved path, never on the caller's spelling, because `sources/` and `CORE-RULES/` walked straight past a literal comparison on a case-insensitive volume and a real Core rule was moved. It also refuses a note under a symlink, a path resolving outside the vault, a destination that already exists (merging two notes is a person's judgement), a file that is not valid UTF-8, and anything already at the cheapest tier. The destination is created with `O_EXCL|O_NOFOLLOW` at mode `0600` and then given the source's mode, so a private note cannot become world-readable and a racing second run cannot leave the note existing nowhere. CRLF survives; nothing rewrites the index, and the preview says exactly that — `MEMORY.md` rows keep pointing at the pointer, which is correct and worth tidying.

These refusals live in the script because `guard_protected_paths` is a **PreToolUse hook**: it sees the model's Write tool and never sees a script writing a file.

Exit codes: report — `0` nothing to report · `1` findings · `2` usage. Demote — `0` done (or previewed) · `1` refused, and why · `2` usage · `3` an error mid-move, with what is where.

`/gt:gt-settings`

View and change what Golden Thread does automatically. Every automatic behaviour is registered here and can be switched off.

SETTING	VALUES	DEFAULT	WHAT IT DOES
<code>component_updates</code>	<code>off</code> · <code>report</code> · <code>confirm</code> · <code>auto</code>	<code>report</code>	At session start, compares installed hooks/scripts against plugin source and reports drift
<code>version_check</code>	<code>off</code> · <code>report</code>	<code>report</code>	At session start, reports when a newer plugin version is checked in than the one installed
<code>orphan_check</code>	<code>off</code> · <code>report</code> · <code>reap</code>	<code>report</code>	At session start, looks for abandoned background Claude workers; <code>reap</code> stops them. A worker is judged by who owns it (0.15.0): a shell with a live <code>claude</code> process above it belongs to that session and is listed as information — session id, claude pid, uptime, and WAITING on: <what it polls> for a poll loop — and is never reaped, even by <code>reap</code> . ORPHAN means no <code>claude</code> remains above it. This session's own undeclared or idle workers are still raised
<code>push_check</code>	<code>off</code> · <code>report</code>	<code>report</code>	At session start, reports vault commits not yet pushed
<code>surface</code>	<code>off</code> · <code>on</code>	<code>on</code>	At session start, shows the MUST DO block from <code><vault>/deadlines.md</code> and every waiting handoff each session, and pre-compaction state files once each (0.17.2). Never writes the vault. See gt surface.py
<code>handoff_surface</code>	<code>any</code> · <code>project</code> · <code>manual</code>	<code>any</code>	Where a handoff that has not been handled is shown (0.17.2): <code>any</code> every session start, whatever project is opened; <code>project</code> only when <code>/gt:gt-open</code> opens the handoff's own project; <code>manual</code> only in <code>/gt:gt-handoff-handle</code> . <code>project</code> needs <code>/gt:gt-open</code> — a session that never opens the project never sees it, which is why <code>any</code> is the default
<code>task_surface</code>	<code>off</code> · <code>on</code>	<code>on</code>	At session start, one line counting <code>p:: 1</code> tasks waiting on you — how many overdue, how many open over a week — with the commands to list and work them (0.17.2). A count, never the tasks; deferred tasks are not counted until their date
<code>protected_paths</code>	<code>off</code> · <code>ask</code>	<code>ask</code>	A Write or Edit to the vault's <code>core-rules/</code> or <code>global-memory/</code> , to <code>~/claude/golden-thread/</code> , or to <code>~/claude/settings.json</code> always shows the permission prompt; editing an existing file in <code>Sources/</code> is refused (supersede it with a new file). Shell commands that write those files are not seen
<code>test_gate</code>	<code>off</code> · <code>warn</code> · <code>auto</code> · <code>block</code>	<code>auto</code>	Refuse a <code>git commit</code> of code whose tests have not been seen to pass; <code>auto</code> blocks only where the repo has a test command
<code>parallel_work</code>	<code>off</code> · <code>on</code>	<code>on</code>	Whether divisible work runs in parallel at all; <code>off</code> also stops the Core rule being injected
<code>parallel_max</code>	<code>auto</code> · a positive integer	<code>auto</code>	Ceiling on concurrent workers. <code>auto</code> = as many as the machine allows

Settings that come from modules. Since 0.15.0 a module declares its own settings in its `module.json` — `key`, `default`, `values`, `summary`, and an optional long `detail` that `gt-settings.py explain <name>` prints. They are listed under `module <name>` in `show` and exist only while the module is installed. A value you set for a module that is later switched off is kept and shown as *OFF — settings kept, not in effect*; it takes effect again after `bash install.sh --with <module>`.

SETTING	MODULE	VALUES	DEFAULT	WHAT IT DOES
report_card	report-card	off · minimal · full	minimal	At <code>/compact</code> and session end, summarises session hygiene; the card is shown at the start of your next session, because output at those two events is never displayed
closeout_check	report-card	off · ask	ask	At <code>/compact</code> and session end, names a project that looks finished so you are asked whether to close it
usage_meter	usage	off · status · login · both	both	Where the plan-allowance meter shows. <code>off</code> still records readings, so turning it back on has a history behind it
usage_alert	usage	early · normal · late · always	normal	How near a ceiling the allowance must be before the meter speaks — or <code>always</code> , to keep it on screen permanently
watch	watch	off · report	off	<code>/gt-watch:gt-watch</code> : the cron fetch and the session-start report of repo changes

report_card (detail):

```
off    nothing
minimal hygiene only -- what went wrong in THIS session  (default)
full   hygiene, plus vault features available and unused

Fires on PreCompact so it is produced while there is still context to write
it in, rather than competing for the last of it at session end.
```

closeout_check (detail):

```
off    nothing
ask    name each project whose signals say it may be finished, with the
       reasons, so the assistant puts the question to you  (default)

Runs the vault's own `Projects/golden-thread/tools/gt_closeout.py`. Signals:
most open tasks past due, most tasks checked off with nothing urgent left,
no new task and no write-back for three weeks, or no open task at all. Every
time the question is put to you it is recorded with the signal values, and
your answer is recorded beside it, so `gt_closeout.py history` can show what
'ready to close' has actually looked like for you and the thresholds can be
tuned to that rather than to a guess.

On 2026-09-05 the CYC26 talk had been delivered for two days while 25 of its
rehearsal tasks sat open and overdue at the top of the rollup, above live
trading work. Nothing had asked.
```

watch (detail):

```

off      no fetching and no report (default)
report   the cron fetch runs, and session start lists unacknowledged upstream
         changes: P0s first with repo and reason, review items one line each,
         routine changes as a count

```

A watch is a note in `Projects/golden-thread/watches/`. The fetch runs from cron (`gt_watch.py install-cron`), never from a session; the session-start hook only reads the local queue in `~/.claude/golden-thread/watch/`. P0 comes from fixed rules -- a security advisory, a CVE or GHSA id, 'security fix' in a release -- never from a reading of commit prose, because a P0 that cries wolf stops being read. `GT_WATCH=off|report` in the environment overrides this setting.

Heads-up: `parallel_work` is `on` and `parallel_max` is `auto` out of the box, so after upgrading to 0.12.4 a long run will use as much of your machine as it can — many processes at once, CPU well above 100%, and the fans to match. That is intended. Cap it with `set parallel_max <N>` or switch it off with `set parallel_work off`; nothing else about the plugin changes.

Test receipts. `core_test_before_commit` needs evidence that the tests passed on *this* tree, so a run leaves a receipt and the commit guard reads it:

```

gt_test_receipt.py record --repo . --what "pytest -q" --ok      # any project
gt_test_receipt.py latest --repo .                             # what was last recorded
gt_test_receipt.py check  --repo . --files a.py b.py           # does a receipt cover these?

```

A receipt covers a file only if it is newer than that file, so editing something after the run invalidates it automatically. `tests/run.sh` and `dev/release-check.sh` record their own. A repo with no tests is exempted once with `touch .gt-no-test-gate`; a single commit with `GT_TEST_GATE=off`.

The machine profile. `parallel_max: auto` resolves through `parallel_profile`, which `install.sh` measures on this machine and re-measures at every upgrade — cores for CPU-bound work, 2x cores (bounded by memory) for I/O-bound. Inspect or refresh it:

```

gt_settings.py detect-machine      # what this machine would measure
gt_settings.py detect-machine --write  # measure and store it

```

Setting a number above what the machine can use is refused at the point you set it, and a number above core count is accepted with a note that it only helps I/O-bound work.

The budget as a number. Any script, in any project, can ask what the settings allow instead of inventing a worker count:

```

JOBS=$(python3 ../scripts/gt_settings.py jobs 42 --io-bound) # 42 units of I/O-bound work
JOBS=$(python3 ../scripts/gt_settings.py jobs 8)              # 8 units, CPU-bound

```

It returns 1 when `parallel_work=off`, never more than `parallel_max`, and never more workers than there are units of work. `gt_settings.parallel_jobs()` is the same answer from Python.

```
python3 $SCRIPTS/gt_settings.py show          # current state of all settings
python3 $SCRIPTS/gt_settings.py explain <name> # reasoning behind the default
python3 $SCRIPTS/gt_settings.py set <name> <value>
```

`component_updates` at `auto` will not overwrite an installed file that is newer than the source, and will not delete a file the source lacks — real drift ran that direction, so a naive "source is truth" updater would silently break enforcement on every session start.

/gt:gt-runbook-lint

Finds procedures duplicated across runbooks and routes them to the right shared layer — `PROTOCOL.md`, a Knowledge page, or a repo `CLAUDE.md`.

/gt:gt-scan

Check a tree against the language definitions **in effect on this machine**, and say which checks ran. What it can check, per language, is in [What /gt:gt-scan can check, and how to ask](#); this is about operating it.

```
python3 $SCRIPTS/gt_scan.py <path> [--vault "<vault>"] [--only language] [--json]
python3 $SCRIPTS/gt_scan.py --list          # the members, and whether each is installed
python3 $SCRIPTS/gt_scan.py <path> --all-files
```

The vault is optional — without one you get the shipped definitions and no local packs.

It is an aggregator over leaf commands, and owns no checking logic. There are two members:

MEMBER	SCRIPT	CHECKS
language	gt_scan_language.py	naming conventions and file encoding, per language
code	gt_scan_code.py	source validation — code against the <code>lint</code> rules in effect

A member is listed only when it exists: a planned-but-unbuilt leaf must never appear, because a listed-but-missing member reads as coverage. `--only` narrows the run, `--all-files` widens it, and `--timeout` bounds a member. `code` has its own section under [Checks and cadences](#) — including the rule that a rule whose `evaluator tier` is absent is reported SKIPPED and never silently passed, which is the same rule as "a member that could not run is not a pass", one level down.

The denominator is the declared set. It prints `running N scan member(s)` before it starts — composition is never a surprise — and a `N of M member(s) ran` line after. A member that is not installed, times out, crashes, or exits for any reason other than clean-or-findings is reported as **not having run**, never dropped from the count, and sets exit `3` even when another member succeeded. Read that line *before* the finding count: "nothing is wrong" and "nothing was checked" print identically otherwise. **A member that could not run is not a pass.**

EXIT	MEANING
0	all members clean
1	findings, and every member ran
2	usage
3	a member could not run

The leaf has one more, which reaches you as an aggregator 3 : exit 4 , *nothing to scan with*. `gt_scan_language.py` refuses to scan when not one definition resolved, rather than walking the tree and reporting nothing — "nothing matched" and "nothing was looked for" are the same output otherwise. When that happens, `gt_registry.py sources` says which packs are missing or failing to verify. Running the leaf directly is worth knowing for the same reason:

```
python3 $SCRIPTS/gt_scan_language.py <path> [--vault "<vault>"] [--only encoding,naming]
```

It prints `definitions in effect: encoding=N, naming=N` — the count it actually loaded, not the count it hoped for.

"Found but not checked" is a gap, not coverage. A construct the scan locates (a `construct` pattern) and then compares against nothing (no `naming` rule) produces no findings and looks exactly like a clean language. `--languages` gives it its own column and a total — *N construct(s) are located and then never checked* — so the gap is stated rather than discovered. Never report a language as checked on the strength of it appearing in the table: read the last column.

A noisy rule is a definition disagreement, not a bug. The scan prints the exact `retract` snippet at the moment the rule annoys you, keyed on the finding's own language. It prints and never writes: a tool that silently edited the pack directory could switch off its own checks, and that directory is guarded to `ask` so a person sees such a change happen. The snippet goes in a pack in **your own vault**, and only local packs may retract — see *Switching a definition off* under [Packs and the registry](#). Never write one for someone without asking; switching off a check is theirs to decide.

Generated, vendored and static files are skipped (the `classify` slot decides which), as are paths in `ignore`, binaries, and anything over 2 MiB. `--all-files` includes the skipped kinds and will produce findings nobody owns — use it deliberately.

This scan prints source text — an identifier *is* the finding — so do not point it at content where the text itself is the sensitive thing. Credential scanning is deliberately a separate tool, and `gt_secrets.py` is now that tool: it is not a member here and must not become one. It lived here once, and a validation on 2026-09-16 found it missed a `.env` file entirely, had no generic `KEY= / SECRET=` rule, and leaked the very values it redacted, because the naming check printed raw source text on the same run. A secret scanner that misses is worse than none, because people stop looking. Two tools with opposite output rules in one process is the whole reason for the split — see [gt_secrets.py](#). If you are checking a tree for credentials, run that; `/gt:gt-scan` does not cover them, and `gt_sweep.py` runs both.

Findings are advice about conventions, not defects. Do not "fix" a repo's naming wholesale because a pack disagreed with it.

`/gt:gt-allin`

Every check in one command, and an honest account of which ones executed.

```
python3 $SCRIPTS/gt_allin.py --vault "<vault>" --repo "<repo>"
python3 $SCRIPTS/gt_allin.py --list          # the members, and whether each is installed
python3 $SCRIPTS/gt_allin.py --only scan,lint # narrow the run
```

It is an **aggregator**, like `/gt:gt-scan`, and owns no checking logic: every member is a command that runs and is tested on its own. The moment the aggregator starts deciding things itself there is a third behaviour nobody tests.

MEMBER	WHAT IT CHECKS	NEEDS
<code>scan</code>	code against the language definitions in effect	-- repo
<code>lint</code>	vault structure: links, orphans, index gaps	-- vault
<code>optimize</code>	vault content that costs context — report only, never applied here	-- vault
<code>doctor</code>	install health: versions, drift, wiring	—
<code>secrets</code>	credential-shaped values in the tree — names, lines, lengths, never values (0.17.2)	-- repo
<code>runbooks</code>	runbook lines duplicated across projects, <code>gt_lint.py --runbooks</code> (0.17.2)	-- vault
<code>wiki</code>	wiki pages: broken links, orphans, stale and unsourced pages — gt-wiki's <code>wiki_lint.py</code> , only while the gt-wiki module is installed (0.17.2)	-- vault
<code>tests</code>	the repo's own test suite, found through the commit guard's discovery table (<code>tests/run.sh</code> , <code>pytest</code> , <code>cargo</code> , <code>go</code> , <code>npm test</code> , <code>make test</code>); a pass records the receipt <code>/gt:gt-allin-commit</code> checks (0.17.2)	-- repo
<code>validations</code>	files changed since their recorded validation, <code>gt_validation.py list</code> (0.17.2)	-- repo

`tests` and `validations` came last (owner, 2026-09-28: *"add all the other checks into allin"*). A repo with no discoverable test command reports `tests` as **could not run**, never as clean. One check is deliberately NOT a member: `gt_code_review.py` plans a review for a model to perform and finds nothing by itself, so as a member it could only ever report "ran, clean" — a pass for work nobody did.

The last three existed and ran nowhere "everything" was run. `gt_secrets` in particular had no aggregator home at all, because it is deliberately not a `/gt:gt-scan` member — a credential finding must never share a report with findings that print source text. `wiki` joins the roster only while its module is installed: someone who switched the wiki module off must not be told "could not run" on every run for a check they chose not to have. So the roster is six members, seven with gt-wiki.

The headline is *N of M members ran*, and the denominator is the declared set. A member that could not run — not installed, or missing the `--repo` or `--vault` it needs — is **reported, never silently skipped**, because a short finding list from a run where half the members failed reads exactly like a clean bill of health and is the opposite. "A member could not run" outranks "a member found something", so report what did not run *first*, then the findings, then a judgement. **A member that could not run is not a pass.**

Exit codes: `0` all clean · `1` findings, every member ran · `2` usage · `3` a member could not run.

It never pushes and never applies anything. There is no flag that makes it push. Push is outward-facing and effectively irreversible — other people fetch it, CI acts on it — and an aggregator is the one place a partial run is easily mistaken for a complete one, so a command that pushed after a run where a member failed would break the Core rule it exists to enforce. `--suggest-push` prints the command *for you to run*, and refuses even that unless every **declared** member ran clean: narrowing the run with `--only` is itself enough to withhold the suggestion, because "everything that ran was clean" was true and misleading while never saying how few ran. `optimize` runs in report mode only, for the same reason — a sweep that edits files as a side effect of "checking everything" changes content without anyone deciding to.

All-in tells you the state; it does not change it. Fix findings with the owning skill, run the tests yourself, and push yourself.

/gt:gt-allin-commit

The deliberate act, kept separate from the sweep. `/gt:gt-allin` reports and changes nothing, so it can be run freely by anyone at any time; this one changes the repository.

```
python3 $SCRIPTS/gt_allin_commit.py --repo "<repo>" --vault "<vault>" -m "your message" --dry-run
python3 $SCRIPTS/gt_allin_commit.py --repo "<repo>" --vault "<vault>" -m "your message"
```

`--dry-run` runs every check and commits nothing — the safe first move, always. It commits **the index, not the working tree**; nothing staged is a refusal, not a no-op. The staged file list is printed *before* the verdict.

It commits but will not push. A commit is local and reversible — `git reset` undoes it and nobody else ever saw it. A push is outward-facing and effectively permanent. That asymmetry is the whole line, and there is no `--push` flag. (After a successful commit it checks whether HEAD is still ahead of its upstream, and says so if not: `git commit` runs the repository's own hooks, and a post-commit hook can push, so a flat "NOT pushed" would be a claim this tool is not in a position to make.)

It checks the test receipt itself, rather than trusting the hook. `guard_test_before_commit` already denies a commit with no passing receipt — but it is a **PreToolUse** hook: it sees the model running `git commit` in a Bash call and never sees a *script* running it through subprocess. A script that shelled out to git would walk straight past the one mechanism enforcing Core rule 4, so the same check is made here, directly, against the same ledger. **Protection cannot depend on who is holding the pen.**

It refuses, naming every reason at once, for any of:

REFUSAL	WAY PAST IT
a check could not run , or crashed rather than reporting	none — an unknown is not a finding, and <code>--allow-findings</code> cannot wave it through
the checks reported findings	<code>--allow-findings</code> , once <i>you</i> have decided they are acceptable
no passing test receipt covers every staged file	run the tests. Editing a file after a run invalidates the receipt, which is the point
you are on the default branch	branch first, or <code>--allow-default-branch</code> . Compared casefolded, against the repo's own default where it has one
a merge, cherry-pick, revert or rebase is in progress	finish it with git yourself, rather than letting this conclude it with a message written for an ordinary commit

Exit codes: `0` committed · `1` refused, and why · `2` usage · `3` a check could not run, or the commit itself failed.

Never pass `--allow-findings` on someone's behalf — accepting a finding is a judgement about this specific change. Never work around a missing receipt; run the tests, because "I ran them" is the claim the rule was built to stop trusting. When reporting a successful

commit, say which tests the receipt covers. The refusals are the feature.

Checks and cadences

Three cadences run the same checks over deliberately different file sets, and the difference is the point:

CADENCE	WHAT IT LOOKS AT	WHAT IT DOES ABOUT IT
tests/run.sh	the file set the scanners define	fails the run
the commit gate	the staged diff only	denies the commit
gt_sweep.py	the whole tree, weekly	reports, never blocks

The narrow two are what keep the gates usable. A commit gate that scanned the whole tree would punish you for someone else's old code, so it looks only at what you are adding — but that means nothing ever re-examines what is already there. A rule added today never sees a file nobody touches, and a credential committed before the gate existed stays committed. The sweep is the cadence that goes back over it, and it reports rather than blocks because there is no commit in front of it to refuse.

gt_secrets.py — credentials in the wrong place

```
python3 $SCRIPTS/gt_secrets.py <path> [--vault "<vault>"] [--staged] [--json]
python3 $SCRIPTS/gt_secrets.py <path> --baseline .gt/secrets-baseline.json
python3 $SCRIPTS/gt_secrets.py <path> --write-baseline .gt/secrets-baseline.json
```

Its output contract is the reason it is a separate tool and not a `gt_scan` member:

Nothing in this process ever prints matched source text, and no other check runs beside it.

A finding is a path, a line, a rule id and a length. Never an excerpt, never a prefix, never a hash. Every other Golden Thread scanner prints the text it found, because there the text *is* the finding; here the text is the thing being protected. Those are opposite output rules, and two tools with opposite output rules in one process is exactly how the 0.16.0 attempt leaked — its `secrets` check printed only a length while its `naming` check, same tool and same run, printed raw source text, so a credential-shaped identifier reached stdout and `--json` in full. That is why this is not a member of `/gt:gt-scan`, and why registering it as one would be a regression rather than a tidy-up.

`--baseline` reports only what is **new**, so an existing finding you have already triaged does not drown the ones that arrived today. `--write-baseline` records the current set — read it before you accept it; a baseline written without looking is a decision to ignore whatever was there.

Exit: `0` clean (nothing new against the baseline) · `1` found something · `2` could not run, or could not scan part of what it was asked to scan. **Silence is never clean**: every successful run ends with an affirmative naming what was covered, because a scanner that could not run prints an empty finding list and so does a clean tree.

gt_scan_code.py — source validation

```
python3 $SCRIPTS/gt_scan_code.py <path> [--vault "<vault>"] [--staged] [--json]
python3 $SCRIPTS/gt_scan_code.py <path> --sarif findings.sarif
python3 $SCRIPTS/gt_scan_code.py --rules          # every rule in effect, and its tier
```

A leaf command, sequenced by `gt_scan.py` as the `code` member alongside `language`. Unlike `gt_secrets.py` it does print source text, because here the text is the finding — which is precisely why the two may never share a process.

Rules are data, never code. Every rule comes from a `lint` pack in the slot registry, in a documented subset of ast-grep's rule schema: `id`, `message`, `severity`, `note`, `files`, `ignores`, and a `rule` object with atomic (`pattern`, `kind`, `regex`), relational (`inside`, `has`, with `stopBy`) and composite (`all`, `any`, `not`) forms. Same names, same nesting, same meanings as upstream, so a user can read ast-grep's published rule reference and have it apply.

Each rule declares the **evaluator tier** it needs, and a tier is either present or it is not:

TIER	EVALUATOR	PRESENT WHEN
text	regex over the file	always
stdlib	Python's own <code>ast</code>	always, for Python
astgrep	the ast-grep CLI	<code>ast-grep</code> ≥ 0.45.3 is on <code>PATH</code>
tree-sitter	tree-sitter grammars	the grammar is installed

A rule whose tier is absent is reported **SKIPPED**, never silently passed. This is the same rule as "a member that could not run is not a pass", one level down: a rule that was never evaluated and a rule that found nothing produce identical output otherwise, and the difference between them is the whole value of the scan. The affirmative line names what was skipped, and SARIF carries it in `toolExecutionNotifications`.

Exit: `0` clean, everything ran · `1` findings (whether or not anything was also skipped) · `2` usage · `3` nothing found **but** a rule was skipped or a pack problem was reported, so the scan does not cover what it was asked to cover · `4` nothing to scan with. `1` outranks `3` deliberately: a caller deciding whether to block needs "was anything found", and a run can both skip a rule and find something. Coverage is reported either way.

The ast-grep CLI is **optional and not bundled**. `brew install ast-grep` or `npm i -g @ast-grep/cli ; install.sh` offers it and never installs it behind your back. Below 0.45.3 it is refused rather than used, because a partial version silently under-matches, which is the failure this whole section exists to prevent.

gt_sweep.py — the weekly cadence

```
python3 $SCRIPTS/gt_sweep.py --vault "<vault>" [--path "<tree>"] [--only secrets,code] [--json]
```

Runs the checks above over the whole tree and **reports**. Members are `secrets` (`gt_secrets.py`) and `code` (`gt_scan_code.py`), each reading `.gt/<member>-baseline.json` from the tree when one is there. `node_modules`, `.venv` and `__pycache__` are excluded by default — measured on gt's own repo, the credential scan went from 48 findings to 10 on that exclusion alone, and a sweep that reports the same finding once per retained release is a sweep nobody reads twice.

Exit: `0` the sweep ran, with or without findings · `2` usage · `3` a member could not run, so the sweep does not cover what it claims to.

`gt_check_report.py` — so a gate leaves a record

```
python3 $SCRIPTS/gt_check_report.py record --vault "<vault>" --check secrets \
    --verdict clean|findings|cannot-run [--count N] [--scope "..."] [--ref HEAD] [--detail TEXT]
python3 $SCRIPTS/gt_check_report.py show --vault "<vault>" [--check secrets] [--json]
```

A gate tells you about the commit in front of you and nothing about the week. Three questions need history and cannot be answered from an exit code: **is this check actually running**, is it getting noisier or quieter, and did anyone ever look at the thing it flagged.

What is recorded is a verdict, a count, a scope and a ref — never a finding's content. `gt_secrets.py` exists because a credential must not reach a log, and a vault file is a file like any other. A report that quoted the finding would undo the tool it is reporting on.

`show` exits `1` when there are no reports yet, which is a different thing from a clean history and reads differently on purpose.

`gt_code_review.py` — the framework, not the opinions

```
python3 $SCRIPTS/gt_code_review.py dimensions [--vault "<vault>"] [--json]
python3 $SCRIPTS/gt_code_review.py plan <path> [--vault "<vault>"] [--staged] [--json]
python3 $SCRIPTS/gt_code_review.py validate <findings.json> --root <path> [--json]
python3 $SCRIPTS/gt_code_review.py report <findings.json> --root <path> --vault "<vault>" [--ledger F]
```

gt does not perform the review, and ships zero review dimensions. It cannot perform one: "does this abstraction earn its keep" has no mechanical oracle, and a tool claiming to test that is testing something else. What gt owns is everything *around* the judgement, which is most of what makes a review trustworthy rather than decorative.

The dimensions are **yours** — `<vault>/Projects/golden-thread/packs/review.<name>.pack.json`, each entry an `id`, a `title`, a `rubric` and optionally a `files` glob. The rubric reaches the plan **verbatim**; a rubric gt paraphrased would be gt's opinion wearing your name. A core review pack would make gt's idea of a good review everyone's default, so there is not one, and a test asserts there never quietly becomes one.

Zero dimensions configured exits `3`, never `0`. Since gt ships none, the empty case is the *default* case — and if it reported success, every user who had never configured a dimension would be told their code had been reviewed. The message names the file to create.

validate is the deterministic half, and rejects any finding that cannot be **checked**: a file that does not exist, a line past the end of one (the reason says how long the file actually is), a dimension nobody configured, a severity outside the vocabulary, or **confirmed: false** — a verification pass that says no must be honoured, or the pass is decoration. Rejections are **counted in the summary**, never silently dropped. **report --ledger** suppresses a finding already declined once: a review that re-proposes what was already dismissed stops being read.

Scope comes from `gt_scan_code.in_scope` rather than a second definition of "what counts as code" — two tools disagreeing about one tree would mean the one nobody runs is the one that was right. An empty scope is an **outcome**, reported as such, never a silent upgrade to reviewing everything.

`gt_schedule.py` — the scheduled jobs, installed by a script rather than by hand

```
python3 $SCRIPTS/gt_schedule.py list
python3 $SCRIPTS/gt_schedule.py install daily --vault "<vault>" [--repo PATH ...] [--hour H] [--minute M]
python3 $SCRIPTS/gt_schedule.py install sweep --vault "<vault>" --repo TREE
python3 $SCRIPTS/gt_schedule.py check daily
python3 $SCRIPTS/gt_schedule.py remove daily
```

JOB	RUNS	WHAT IT DOES	NORMAL EXITS
daily	22:00	<code>gt_daily.py</code> — the day's facts into <code>Daily Notes/<today>.md</code>	<code>0</code> , <code>1</code> (nothing to report)
lint-weekly	Mondays 07:00	vault + wiki lint, reported into the vault	<code>0</code>
sweep	Mondays 07:30 (0.17.2)	<code>gt_sweep.py</code> — whole-tree secrets + code sweep, verdict filed in the vault. Takes <code>--vault</code> and exactly one <code>--repo</code>	<code>0</code>

A scheduled job runs the installed copy in `~/ .claude/golden-thread/hooks/` — a script under CloudStorage fails under `launchd` — and `install` refuses a job whose script is not there. Until 0.17.2 `gt_daily.py` was not installed there, so the `daily` job could never be installed, and `gt_sweep.py` promised a `launchd` job that did not exist. Both install there now, with the sweep's members and their imports, and the release gate fails any job whose script does not reach the hooks dir.

`install sweep` currently refuses — a known limitation. Before scheduling, `install` runs the job's own `--check` ; for the sweep that proves each member can run from the hooks dir, over an empty tree. Today it cannot: `gt_registry` looks for `packs/` beside itself and there is none there, so every member would report "could not run" every Monday. Refusing to install is the check doing its job.

The weekly lint agent was installed **by hand** in 2026-09-08, and its only record was two lines in a runbook: a manual step with no `--check` and no rollback, on a machine where scheduled jobs fail in ways a terminal run cannot reproduce. This retrofits the standing rule that any human step becomes a script with `--check` , self-proof and rollback.

`check` validates through `launchd`, not through a terminal run, and reads back `launchd`'s own *last exit code*. That distinction matters on macOS, where a job can run perfectly in your shell and be denied the filesystem under `launchd`. It also knows which exit codes are a **normal outcome** for each job, because a check that reports a normal result as a failure is worse than no check — people stop reading it.

That table was wrong for `lint-weekly` until 0.17.2, in the other direction. It listed `1` as normal on the belief that the job exits `1` when the lint finds something. It never did — `gt_lint_weekly.py` returns `0` on every path it chooses — so the only way it exited `1` was an uncaught exception, and a crash read as "findings". Now only `0` is normal; `gt_lint_weekly.py` exits `3` and writes **COULD NOT RUN** into its report when a lint died, instead of "Findings: 0" built from two tracebacks. **A benign exit code must be one the script returns on purpose, never Python's crash code.** `gt_daily.py` turns an uncaught exception into `3` for the same reason.

Known and not fixed: macOS refuses calendar-fired runs the vault. Every calendar-fired `lint-weekly` run on record (09-14, 09-21, 09-28) was refused by macOS privacy controls (TCC) reading existing files in a vault under CloudStorage; only runs kicked from a session succeeded. The `daily` job will very likely fail the same way. The doctor's `schedule` check reports it; the fix — a privacy grant, a vault outside CloudStorage, or no calendar trigger — is the machine owner's choice.

gt_daily.py — the day's facts, and nothing else

```
python3 $SCRIPTS/gt_daily.py --vault "<vault>" [--date YYYY-MM-DD] [--repo PATH ...] [--dry-run] [--check]
```

Writes tasks closed, commits per repo, event counts, wiki item **counts**, and an active span per project into one fenced, clearly-marked block in `Daily Notes/<date>.md`, replaced whole on each run.

Terse by design. It does not explain, summarise or interpret. The owner writes the meaning; a generated block that editorialised would encode a reading of the day that is not theirs. For the wiki it records how many items and how much time, never what they said.

It **never touches a line outside its block**, and specifically never touches `## Noticed` — that is the unfiled capture surface `/gt:gt-review` sweeps, and a tool tidying it would defeat the sweep. Exit `1` means nothing to report for the day, which is not an error; a crash exits `3` (0.17.2), never `1`. `--check` verifies every dependency and exits; `--dry-run` prints the block and writes nothing.

gt_state.py — write the state before the context runs out

```
python3 $SCRIPTS/gt_state.py check [--margin N] [--write] [--json]
python3 $SCRIPTS/gt_state.py write [--reason R]
python3 $SCRIPTS/gt_state.py show
python3 $SCRIPTS/gt_state.py hook      # PreCompact / SessionEnd: the backstop
```

Three pieces existed and nothing joined them: `gt_usage` reads the context percentage and only *displays* it; `gt_report_card` runs at `SessionEnd` and `PreCompact` and writes a **health summary** — uncommitted files, stale rollup — with nothing about what the session was doing; and `gt_handoff` gathers exactly the right content but is manual, so nothing ever calls it. This is the join, and it needs nobody to think of it.

The signal is `ctx_pct`, and the distinction is the whole design. `readings.jsonl` carries context fill and rate-limit allowance side by side, and they are unrelated. A real reading from the session that prompted this:

```
{"five_hour": 5, "seven_day": 10, "ctx_pct": 91}
```

5% of the five-hour allowance, 91% of the context. Triggering on the allowance meter would have fired at entirely the wrong moment **and looked correct doing it**, because both are percentages with plausible values. A test asserts the tool never reads `rate_limits` at all.

It fires at `--margin` (default 5) below the flag point for your `usage_alert` mode, **once per crossing** rather than on every turn, and is wired to `UserPromptSubmit`. `PreCompact` is the **backstop, not the primary** — it fires when compaction is already starting, so the write competes with the thing it exists to survive — and its file says which one wrote it, so nobody reads a backstop write as the early one having worked.

A missing or unreadable usage ledger is reported as **"cannot tell"**, never as room to spare: the usage module may not be installed at all, and silence from a source that was never there must not read as a measurement saying everything is fine. Every failure path exits `0`; a state write that broke a turn would be worse than the gap it closes.

gt_surface.py — what the last session left, shown to the next

```
python3 $SCRIPTS/gt_surface.py check [--vault V] [--dry-run] [--json] # what SessionStart runs (with --hook)
python3 $SCRIPTS/gt_surface.py must-do [--vault V] # the MUST DO block alone
python3 $SCRIPTS/gt_surface.py handoffs --project SLUG [--vault V] # one project's waiting handoffs (/gt:gt-open)
```

By 0.17.1 gt could write everything a next session needs and put none of it in front of that session. `gt_handoff.py` wrote handoffs, `gt_state.py` wrote state files, `/gt:gt-work` raised tasks citing a handoff, the owner kept dated must-do items — and, measured on 2026-09-28, none of the `SessionStart` hooks read a handoff, a state file, `TASKS.md` or any `p::`. Two credential rotations sat 15 days overdue at the vault's top priority, worked around almost daily, because **priority is a sort order, not an alarm**: sixty tasks already sat at the top tier across ten projects, and one more alerts nobody.

Surfacing is the alarm; the task is the record. This hook is the alarm. It never creates, edits or closes a task, and it never writes to the vault.

What it shows, how often, and the settings that control each are under [What you see at session start](#); the file format is [deadlines.md format](#). Handoff state comes from `gt_handoff_status.py` (see [/gt:gt-handoff-handle](#)); the task line from the vault's `gt_task.py` count (see [/gt:gt-task](#)).

State files are still "once each", and that needs memory: a machine-local "already shown" ledger at `~/.claude/golden-thread/surface/seen.json`, the one thing this hook writes. `--dry-run` leaves it alone.

A clean run says so — `nothing waiting`, in one line — because silence reads the same as a hook that never ran. It is never fatal: any failure exits `0`, since a `SessionStart` hook that broke startup would be worse than the gap it closes. Switch it off with `gt_settings.py` set surface off.

Why this is core and not the report card. The request that led here proposed a deadline check in the report-card module. An alarm must not depend on an optional module being installed, and the report card shows what the *previous* session computed at its end, where a countdown has to be computed live at every start.

Session identity — which machine wrote a claim

`gt_session.py` claims a file for a session, and the `guard_session_claims` hook refuses a write to a file another live session holds (Core rule 1). "Live" is judged by the recorded `pid` — but a `pid` only means something on the machine that recorded it. So every reader first asks: *was this written here?*

Until 0.17.2 the answer was "the recorded host equals this hostname". A hostname is not an identity. On 2026-09-28 a laptop moved to a wired network, DHCP gave it a new name, and every claim written under the old one stopped being this machine's. Nothing failed: the `pids` simply stopped being judged, and each claim would lapse on the heartbeat clock while its session was still running — Core rule 1, disarmed by a DHCP lease, with nothing announced.

Identity is now a machine id: a `uuid4` written once to `~/.claude/golden-thread/machine-id`, created atomically on first use (two first runs cannot mint two ids), never overwritten, never synced between machines. Registrations record it as `machine:`; `host:` stays as a readable label only. The claim guard, `gt_session.py` and `gt_workers.py` all apply one rule (`gt_paths.same_machine`; the vault tool carries a pinned copy, because a vault tool cannot import from the plugin):

THE RECORD HAS...	VERDICT
a machine id, and so do we	the ids decide. Same id under a new name is a rename — reported once, claims still hold
no machine id (written by <code>gt ≤ 0.17.1</code>)	ours while its host label matches, exactly as before. If it does not match, nothing can prove whose it is: its heartbeat decides, and that is said . It adopts the id the next time its own session writes it
a machine id, and ours is unreadable	the host label, as before — and the missing id is announced

Two OS logins on one Mac are two machines now. The id lives under each HOME, beside `workers.jsonl`, which is machine-local by the same design. Each login judges the other's sessions by heartbeat, the cross-machine rule, where before they shared a hostname and judged each other's pids.

A pid no longer vouches for another session. One Claude Code process hosts successive sessions, so an older session's file recording *our own* pid looked live for as long as we ran, and its claims never lapsed. When the recorded pid is the asker's and the session is not, the pid proves nothing about that session, and its heartbeat decides.

/gt-watch:gt-watch

Module `watch` (plugin `gt-watch`), installed by default; it was `/gt:gt-watch` before 0.15.0. `bash install.sh --without watch` removes it, including the crontab line it installed.

Watch any git repo you depend on, and hear about it when it changes in a way you said you care about — raised as a **P0** when it ships a security fix.

```
/gt-watch:gt-watch add <git-url>    - create a watch (any git URL: GitHub, GitLab, self-hosted, file://)
/gt-watch:gt-watch list              - every watch and its last change
/gt-watch:gt-watch check             - fetch now instead of waiting for cron
/gt-watch:gt-watch show <slug>      - Claude reads the new commits and explains them
/gt-watch:gt-watch ack [<slug>]     - mark changes seen
/gt-watch:gt-watch remove <slug>    - stop watching
```

A watch is a note — `Projects/golden-thread/watches/<slug>.md`, editable in Obsidian's Properties panel. Its frontmatter says what to watch: `url`, `track` (commits, tags, releases), `branch`, `watch_paths`, `current_version`, your own `p0_when` patterns, and `starred`.

How it runs. A cron job (`gt_watch.py install-cron --every 1h`) fetches every watch without Claude, classifies each change, and queues it. At session start a hook reads the queue — P0s first, spelled out; review items one line each; routine changes as a count. It never fetches. Machine state lives in `~/.claude/golden-thread/watch/`, outside the vault.

When it is a P0 — decided by rules, never by a model reading commit prose: a new security advisory for the repo; a new tag or release naming a CVE or GHSA id, or saying "security fix/release/update" or "vulnerability"; a commit message carrying a CVE or GHSA id; a change to `SECURITY.md` or your `watch_paths` that says the same; or anything matching your own `p0_when`. A new release, a major-version jump, a change under `watch_paths`, or "BREAKING CHANGE" is a **review** item. Ordinary words like "auth" or "security" in a commit message are not enough — a P0 that cries wolf stops being read.

The module is on by default, but it fetches and reports nothing until you switch it on: setting `watch (off · report)`. `add` offers to switch it on and install the cron entry.

/gt-demo:gt-demo

Module `demo` (plugin `gt-demo`), installed by default. `bash install.sh --without demo` removes it; `--with demo` brings it back.

A guided tour of Golden Thread on PizzaBot 3000, a fictional pizza-ordering project: nine core acts, plus one act for each installed module that ships one — in 0.16.2 `wiki` , `watch` and `flow` , so twelve with the default modules. It runs in its own throwaway vault, so nothing it does can reach yours.

```
/gt-demo:gt-demo start    - build the demo vault and print the command that opens it
/gt-demo:gt-demo tour     - (in the demo session) run the tour; you only click Next
/gt-demo:gt-demo end      - every commit and file the tour produced
/gt-demo:gt-demo clean    - delete the demo vault and build a fresh one
/gt-demo:gt-demo remove   - delete the demo vault; `bash install.sh --without demo` removes the module
/gt-demo:gt-demo status   - is there a demo vault, and how old is it
```

Running it: `start` builds the vault at `~/.claude/golden-thread/demo-vault` and prints `cd <demo vault> && GT_VAULT=<demo vault> GT_WATCH=report GT_WATCH_STATE=<demo vault>/.demo/watch claude` — the extra variables let the watch act run without touching this machine's watch settings or state. Run that in a new terminal and type `/gt-demo:gt-demo tour` . For each act Claude says one line to the audience, does the work with the real skill, says what just happened, and offers buttons — **Next**, **Repeat this act**, **Skip ahead**, **End tour**.

The core acts: Core rules enforced (a prepared reply carrying a key is blocked) · open a project · "what's next?" from the task rollup · capture a finding mid-session · route a stray idea · validate a wrong claim · lint finds a planted broken link · promote a finding to a Knowledge page (which fixes the link) · file the inbox, close the session, and show the receipt. They are plain text in `templates/demo-pizzabot/tour.md` .

Module acts come from each installed module's `module.json` → `demo` file and are placed before the closing act: **wiki** — ingest a source and query the wiki; **watch** — watch an upstream library and see its security release open as a PO; **flow** — render the tour's events as a redacted flow view. A module that is not installed has no act. A module act names its scripts as `<module:NAME>` , which the demo resolves with `gt_demo.sh module-scripts NAME` (exit 3: not installed, the act is skipped).

Why its own vault: a fuller tour writes to shared files (`INBOX.md` , `TASKS.md` , `log.md` , `Knowledge/`) that other sessions also write. Undoing that in a real vault means rewinding history everyone shares. In a throwaway vault, `clean` just rebuilds it. The demo never writes your `vault-config.json` : the demo session is pinned with `GT_VAULT` , which every skill and hook honors. `clean` and `remove` delete a directory only if it carries the demo's marker file.

/gt-flow:gt-flow

Module `flow` (plugin `gt-flow`), new in 0.15.0, installed by default.

Draws how knowledge moved through the vault over time, as **one self-contained HTML file** that opens offline — no network, no CDN. One lane per project, time left to right, each lane split into the ladder's levels (memory → research/decisions/design → Knowledge → global-memory → Core); an arrow joins the level an item left to the level it reached, so a promotion is an arrow climbing. It reads `Projects/golden-thread/events.jsonl` and never writes the vault.

```
FLOW=~/.claude/plugins/cache/golden-thread-plugin/gt-flow/0.17.6/scripts
python3 $FLOW/gt_flow.py render --vault <vault> [--out FILE|DIR] [--redact] \
  [--project <slug> ...] [--since YYYY-MM-DD] [--tasks]
```

- **--redact** before the page leaves your own screen — an Artifact, a chat, a ticket, a screenshot. Every project, path, task id, domain and session becomes a short salted hash and notes are dropped; levels, kinds and counts stay. The salt is random per render, so two redacted files cannot be joined. If the redaction self-check fails, nothing is written.
- Task events (`task.open` , `task.done`) are hidden at first — on a real vault they are most of the stream. They are one click away in the Kinds filter; **--tasks** shows them from the start.
- **--project** is repeatable and includes sub-projects; **--since** takes a date or an ISO-8601 instant.
- Without **--out** the file lands in the current directory (a temp directory when that is inside the vault). An **--out** inside the vault is refused.

Exit codes: `0` written · `1` bad arguments, an invalid event file or an unknown schema version · `2` no events yet (run `gt_events.py backfill --dry-run` to see what history would be recovered) · `3` the filters matched nothing.

Where does this fact go?

THE FACT	GOES TO
"This preset is wrong right now"	<code>memory/</code>
"This API rejects plural keys"	<code>research.md</code>
"We use POST for all mutations, and why"	<code>decisions.md</code>
"The system is structured like this now"	<code>design.md</code>
"This platform behaviour bit two projects"	<code>Knowledge/</code>
"Always diff before overwriting prod"	<code>global-memory/</code>
"How to restart the prod backtest"	<code>runbook.md</code>
"Never scp without backing up"	<code>PROTOCOL.md</code>
"This code lives on host X at path Y"	<code>source.md</code>
"Which box serves which role"	<code>INFRASTRUCTURE.md</code>

Two failure modes, opposite directions: **too narrow** buries a platform truth in one project, so the next rediscovers it the hard way; **too wide** promotes a quirk into `global-memory/` , where it loads into unrelated sessions as a rule that isn't true there. When unsure, file narrow — promotion is cheap, demotion isn't.

/gt:gt-route applies this table for you, against the material actually in hand, without your having to hold the table in your head mid-session. Reach for it when the answer is not obvious — including when the honest answer is that the fact belongs to a project other than the one loaded.

Script reference

```
SCRIPTS=~/.claude/plugins/cache/golden-thread-plugin/gt/0.17.6/scripts

python3 $SCRIPTS/vault_init.py fresh --vault ~/my-vault --domain "My Team"

python3 $SCRIPTS/vault_init.py create-project --vault ~/my-vault \
  --name my-project --title "My Project" --tags "backend,api" \
  --domain platform --topology bastion-direct \
  --repo-url git@github.com:me/my-project.git --runbook

python3 $SCRIPTS/vault_init.py create-project --vault ~/my-vault \
  --name sub-feature --parent my-project --domain platform

python3 $SCRIPTS/vault_init.py connect --vault ~/existing-vault

python3 $SCRIPTS/vault_init.py install-core-rules --vault ~/my-vault

python3 $SCRIPTS/gt_ingest.py ~/Projects/my-project --json

python3 $SCRIPTS/gt_lint.py ~/my-vault --queue ~/my-vault/review-queue.md

python3 $SCRIPTS/gt_settings.py show
```

`gt_tasks.py` runs from the vault, at `Projects/golden-thread/tools/`, beside `gt_adr.py`, `gt_closeout.py`, `gt_edits.py`, `gt_events.py`, `gt_log.py`, `gt_session.py`, `gt_spool.py` and `safe_write.py`. All nine ship as templates and are seeded there by `vault_init.py fresh` or `connect` (what `/gt:gt-init` runs); `install.sh` refreshes copies that predate the installed template. It regenerates `TASKS.md` from every project's `## Tasks` section.

```
python3 ~/my-vault/Projects/golden-thread/tools/gt_tasks.py
```

It infers the vault from its own location, so it needs no arguments; `--vault` is there to override that. **Re-run it before reading `TASKS.md`** — project priority is computed against the clock (stale-P1 ageing, deadline windows, `pp_escalate`), so the ranking changes with time even when no file has changed. `TASKS.md` is generated and must never be hand-edited.

`--json` prints the ranked rollup to stdout without writing `TASKS.md`. Deadline windows may span days (0.13.0): `Fri 16:00 → Sun 16:44 America/Chicago → 0`, including spans that wrap the week; the zone always comes from the rule, never the machine.

`gt_lint.py --json` emits findings as JSON, and `gt_lint.py --runbooks` reports lines duplicated across projects' runbooks — the detection step of `/gt:gt-runbook-lint`.

Troubleshooting

"vault-config.json not found" — run `/gt:gt-init`.

A rule in `CLAUDE.md` seems ignored — it is read at session start. Added mid-session, it applies from the *next* session. Same for hooks in `settings.json`: open `/hooks` once or restart.

Memory files aren't being read — that is intended. `gt-open` reads `MEMORY.md` and loads files on demand. Ask for one by name.

A memory file is never offered — run `/gt:gt-lint` ; it is probably `memory-unlisted` , so nothing knows it exists.

`/gt:gt-query` can't find a page you added — likely `index-gap` .

Lint suppression isn't working — paths are matched case-insensitively against the finding path; make sure you used the path as the finding reports it.

"conflict: vault-config.json points to a different vault" — run `vault_init.py connect --vault <new-path>` .

A scheduled job fails, or `/gt:gt-doctor` 's `schedule` check says FAIL — read the job's `.err` log the check names.

`PermissionError` on a vault under CloudStorage from a calendar-fired run is the known macOS privacy refusal (0.17.2 reports it; it does not fix it): a run kicked from a session succeeds where the scheduled one is refused.

`gt_schedule.py install sweep` refuses — expected in 0.17.2. Its pre-flight proves every member can run from the hooks dir, and they cannot yet find their packs there.

"no machine id" or "hostname changed" notices — see [Session identity](#). A rename is harmless and reported once; a legacy file on a host that no longer matches keeps its claims only while its heartbeat is fresh.

A handoff keeps being shown — that is intended until it is dealt with: run `/gt:gt-handoff-handle` , mark it handled, or defer it to a date. To see waiting handoffs only in their own project, `gt_settings.py set handoff_surface project` .

No **GOLDEN THREAD surface:** line at session start — the hook is unwired or `surface` is `off` . Run `/gt:gt-doctor ( wiring )` and `gt_settings.py show` . A clean run still prints "nothing waiting".

core-unenforced finding in lint — Core rules are stored but not being asserted. Run `vault_init.py install-core-rules --vault <vault>` or wire the hooks by hand. Do not suppress this finding.